VISVESVARAYA TECHNOLOGICAL UNIVERSITY

BELAGAVI–590018

2026-2027

A Community Project Report
On

"Digital Inventory and Sales Management System"

Submitted in the partial fulfillment of the requirements for the 3rd Semester
A Community Project - 1BCP308 for the award of the degree of

Bachelor of Engineering
In
Department of Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

Submitted By

KEERTHANA P	-	**1GV25CI021**
L MOHAMMED FAHAD	-	**1GV25CI023**
MEHFUZA KHANUM	-	**1GV25CI026**
MOHAMMED SAAD	-	**1GV25CI028**

Under the Guidance of

Prof. APOORVA D
Assistant Professor
Dept of CSE (AI & ML)
Dr. TTIT, KGF

Department of Computer Science and Engineering
(Artificial Intelligence and Machine Learning)
Dr. T. THIMMAIAH INSTITUTE OF TECHNOLOGY
Oorgaum, Kolar Gold Fields 563120

Accredited by NBA (CSE, MINING, ECE) and NAAC with 'A' Grade

DR. T. THIMMAIAH INSTITUTE OF TECHNOLOGY

Oorgaum, K.G.F 563120

Accredited by NBA (CSE, Mining, ECE) and NAAC with 'A' Grade

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

CERTIFICATE

This is to certify that the Community Project entitled "Digital Inventory and Sales Management System" is a bonafide work carried out by KEERTHANA P-1GV25CI021, L MOHAMMED FAHAD-1GV25CI023, MEHFUZA KHANUM-1GV25CI026, MOHAMMED SAAD-1GV25CI028 in the partial fulfillment for the completion of 3rd semester of Bachelor of Engineering in Computer Science and Engineering (Artificial Intelligence and Machine Learning) of the Visvesvaraya Technological University, Belagavi during the academic year 2026-2027. It is certified that all corrections/suggestions indicated for the assessment have been incorporated in the Community Project Report deposited in the department library.

The report has been approved as it satisfies the academic requirement in respect of Community Project Report – 1BCP308 prescribed for the Bachelor of Engineering Degree.

.....	.....	.....
Signature of Guide	**Signature of H.O.D**	**Signature of Principal**
Prof. APOORVA D	**Dr. LEELAVATHY S R**	**Dr. SYED ARIFF**
Asst. Prof.	**Assoc. Prof. & Head**	**Principal**
Dept. of CSE (AI & ML)	**Dept. of CSE (AI & ML),**	**Dr. TTIT, KGF**
Dr. TTIT, KGF	**Dr. TTIT, KGF**	

DECLARATION

We here by declare that the Project work entitled **"Digital Inventory and Sales Management System"** submitted to **Visvesvaraya Technological University**, Belagavi during the year **2026-27** in the partial fulfillment for the award of **Bachelor of Engineering** in **Computer Science and Engineering (Artificial Intelligence and Machine Learning)** is a record of bonafide work carried out by us under the supervision of **Prof. APOORVA D, Asst. Prof** Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Dr. T Thimmaiah Institute of Technology – KGF.

We further declare that the work reported in this Project report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: KGF

Date:

Signature of the Candidate

.....	.....
KEERTHANA P	**L MOHAMMED FAHAD**
1GV25CI021	**1GV25CI023**
.....	.....
MEHFUZA KHANUM	**MOHAMMED SAAD**
1GV25CI026	**1GV25CI028**

ACKNOWLEDGEMENT

The successful completion of any task would be incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown effort with success.

It is with the profound feeling of gratitude we would like to express our sincere thanks to our institution **Dr. T. THIMMAIAH INSTITUTE OF TECHNOLOGY, KGF,** for providing excellent infrastructure for successful completion of the Community Project.

We wish to express a whole hearted thanks to our Principal, **Dr. SYED ARIFF, Dr. TTIT, K.G.F,** for being a constant support in carrying out this project.

We would like to extend heartly thanks to our HOD, **Dr. LEELAVATHY S R, Associate Professor and Head, Dept. of CSE (AI & ML), Dr. TTIT, K.G.F,** for being a constant support of encouragement to carry out the Community Project successfully.

We would like to extend hearty thanks to our guide **Prof. APOORVA D, Asst. Professor** and our Coordinator **Prof. MUKESH M, Asst. Professor, Dept. of CSE (AI & ML), Dr. TTIT, K.G.F,** for their valuable suggestions, guidance and support in the completion of this Community Project work.

We would also like to thank all teaching and non-teaching staff of Dept. of CSE (AI&ML), Dr. TTIT, K.G.F who were directly and indirectly supported for carrying out this Community Project successfully.

We extend our hearty thanks to our parents for what we are today and friends who helped us in successful completion of our Community Project report and for the betterment of our career.

KEERTHANA P - 1GV25CI021
L MOHAMMED FAHAD - 1GV25CI023
MEHFUZA KHANUM - 1GV25CI026
MOHAMMED SAAD - 1GV25CI028

ABSTRACT

Small and medium retail stores in India still manage a large part of their business with paper stock registers, spreadsheets and stand-alone billing machines. Stock figures drift away from what is actually on the shelf, reorders are decided by guesswork, and preparing GST-compliant invoices and tax summaries takes considerable manual effort. This project presents a **Digital Inventory and Sales Management System**, a web application that brings the catalogue, stock, purchasing, billing and reporting of a retail store into one place, and adds artificial intelligence to help the owner decide what to reorder and when.

The system is built on the MERN stack with TypeScript: a React single-page application, a REST API on Node.js and Express, and a MongoDB database run as a replica set so that every sale, purchase receipt and stock adjustment is saved as one all-or-nothing transaction. Every change in stock is recorded in a stock ledger, so the stock of a product always equals the sum of its recorded movements and can never become negative. The system follows Indian GST rules: products carry HSN codes and GST rates, supplier GSTINs are verified with their check character, money is stored in paise to avoid rounding errors, and tax is split into CGST and SGST for sales within the state or charged as IGST across states. Access is controlled by roles (admin, manager and cashier) using short-lived JSON Web Tokens and rotating refresh tokens.

Two AI features are designed for the system. The first is a **human-in-the-loop restocking agent**: a demand-forecasting engine based on Holt-Winters exponential smoothing with weekly seasonality and festival uplift estimates each product's demand over its supplier's lead time, computes a reorder point with safety stock, and prepares a restock plan listing the items and quantities to order. The owner reviews and edits the plan, and purchase orders are placed only after the owner confirms. The second is an **"Ask your Inventory" assistant** built on Google Gemini, which answers questions about the shop in plain English by calling the system's own services and can prepare restock plans on request, but can never confirm an order by itself.

At the time of this report the user management, catalogue, stock ledger, stock adjustment and purchase order modules are complete and verified by 235 automated tests, all passing. Point-of-sale billing, reports and the AI modules have been designed and are being implemented. A deterministic simulation of one year of trading (11,870 invoices) with weekday, seasonal and festival patterns is used as the demonstration and forecasting dataset.

Keywords: Inventory Management, Point of Sale, GST Invoicing, MERN Stack, MongoDB Transactions, Stock Ledger, Demand Forecasting, Holt-Winters Exponential Smoothing, Reorder Point, Human-in-the-Loop AI, Large Language Models, Google Gemini.

TABLE OF CONTENTS

DETAILS		PAGE NO
ACKNOWLEDGEMENT		**I**
ABSTRACT		**II**
CONTENTS		**III**
LIST OF FIGURES		**V**
LIST OF TABLES		**VII**
CHAPTER NO.	**DESCRIPTION**	**PAGE NO.**
CHAPTER 1	**INTRODUCTION**	1
	1.1 Motivation	1
	1.2 Objectives	2
	1.3 Scope of the Project	2
	1.4 Applications	3
CHAPTER 2	**LITERATURE SURVEY**	4
	2.1 Comparison Table	7
	2.2 Key Findings	7
CHAPTER 3	**PROPOSED SYSTEM**	9
	3.1 Problem Statement	9
	3.2 System Architecture	9
	3.3 Methodology	11
	3.4 Data Flow	15
	3.5 Use Case Diagram	18
	3.6 Database Design	19
CHAPTER 4	**SYSTEM REQUIREMENTS**	21
	4.1 Software Requirements	21
	4.2 Hardware Requirements	22
	4.3 Functional and Non-functional Requirements	22
	4.4 Datasets	23
	4.5 Sample Code	23
CHAPTER 5	**DATA VISUALIZATION**	29
CHAPTER 6	**IMPLEMENTATION**	34
	6.1 Introduction	34
	6.2 Implementation Details	34

	6.3 Block Diagram	37
	6.4 System Work Flow	37
CHAPTER 7	**TESTING AND RESULTS**	40
	7.1 Test Cases and Execution	40
	7.2 Results	41
	7.3 Login Page	42
	7.4 Product Catalogue	43
	7.5 Inventory	45
	7.6 Purchase Orders	47
	7.7 Suppliers, Users and Role-based Access	49
	7.8 Result Summary	51
CHAPTER 8	**CONCLUSION AND FUTURE ENHANCEMENT**	52
	REFERENCES	54

LIST OF FIGURES

SL. NO	DESCRIPTION	PAGE NO
Fig 3.1	System Architecture	10
Fig 3.2	Development Methodology	11
Fig 3.3	AI Restocking Methodology (Human in the Loop)	12
Fig 3.4	Data Flow Diagram – Level 0 (Context Diagram)	16
Fig 3.5	Data Flow Diagram – Level 1	17
Fig 3.6	Use Case Diagram	18
Fig 4.1	Express Application Setup	24
Fig 4.2	Product Schema (MongoDB / Mongoose)	25
Fig 4.3	GST Calculation (CGST + SGST or IGST)	26
Fig 4.4	Stock Ledger – Atomic Stock Update	26
Fig 4.5	Weighted-Average Cost on Goods Receipt	27
Fig 4.6	Authentication and Permission Middleware	27
Fig 4.7	Role Permission Matrix in Code	28
Fig 5.1	Monthly Sales Revenue	29
Fig 5.2	Daily Sales with Festival Days Marked	30
Fig 5.3	Average Sales by Day of the Week	30
Fig 5.4	Weekly Demand Patterns of Four Products	31
Fig 5.5	Sales Revenue by Category	31
Fig 5.6	Top 10 Products by Revenue	32
Fig 5.7	Share of Sales Value by Payment Method	32
Fig 5.8	Current Stock Status of Active Products	32
Fig 6.1	Block Diagram of the System	37
Fig 6.2	Purchase Order Life Cycle	37
Fig 6.3	Sequence Diagram – Receiving Goods	38
Fig 6.4	Restock Plan States (Design)	38
Fig 6.5	Sequence Diagram – AI Restock with Owner Confirmation (Design)	39
Fig 7.1	Automated Test Run	41
Fig 7.2	Login Page	42
Fig 7.3	Products List	43
Fig 7.4	Add Product Form with Live Price Breakdown	44
Fig 7.5	Product Details with Recent Stock Changes	45
Fig 7.6	Inventory Overview	45
Fig 7.7	Stock History (Stock Ledger)	46

Fig 7.8	Stock Adjustment	46
Fig 7.9	Purchase Orders List	47
Fig 7.10	New Purchase Order with Live GST	48
Fig 7.11	Purchase Order Details	48
Fig 7.12	Receiving Goods	49
Fig 7.13	Suppliers	49
Fig 7.14	User Management (Admin)	50
Fig 7.15	Cashier's View of Products	50

LIST OF TABLES

SL. NO	DESCRIPTION	PAGE NO
Table 2.1	Comparison of Existing Approaches and the Proposed System	7
Table 3.1	Role Permission Matrix	18
Table 3.2	Database Collections	19
Table 4.1	Technology Stack	21
Table 5.1	Summary of the Demonstration Dataset	33
Table 6.1	Implementation Status of the Modules	36
Table 7.1	Test Cases and Execution	40
Table 7.2	Summary of Automated Tests	42

CHAPTER 1

INTRODUCTION

Retail is one of the largest sources of employment and economic activity in India, and the great majority of retail outlets are small, independently run stores: general and grocery stores, supermarkets, stationery and electronics shops, and similar businesses. Such a store typically stocks a few hundred to a few thousand products, buys from a handful of regular suppliers, and serves walk-in customers at one or two billing counters. Its success depends on having the right goods on the shelf at the right time, without locking up money in stock that does not sell.

In many of these stores, inventory and sales are still managed manually. Stock is recorded in registers or spreadsheets, purchases are ordered by looking at the shelves, and bills are written by hand or produced by a billing machine that is not connected to the stock records. Over time the recorded stock drifts away from what is actually in the store, items run out without warning, slow-moving goods accumulate unnoticed, and the owner has little reliable information on which to base decisions. Since the introduction of the Goods and Services Tax (GST) in 2017 and its rationalisation into fewer slabs in September 2025, stores must also issue tax invoices that show HSN codes, the correct split of CGST and SGST or IGST, and the shop's GSTIN, and must keep records that support their tax filings.

This project develops a **Digital Inventory and Sales Management System**: a web application that replaces the stock register and stand-alone billing with one integrated system. The owner and staff sign in from any browser according to their role. Managers maintain the product catalogue and suppliers, raise purchase orders and record deliveries; cashiers bill customers at the point of sale; and every movement of goods in or out of the store is recorded in a stock ledger, so that the stock shown on screen always matches the history of what was bought, sold, returned or written off. GST is calculated automatically from each product's rate and the location of the supplier or customer.

On top of this accurate record of sales and stock, the system adds artificial intelligence where it gives the owner the most value: deciding **what to reorder and how much**. A demand-forecasting engine learns each product's sales pattern, including the weekly cycle, seasons and festivals, and prepares a restock plan before items run out. The owner reviews the suggested items and quantities and confirms the plan before any purchase order is placed. An AI assistant based on Google Gemini lets the owner ask questions such as "What sold most this week?" or "Can you restock based on previous demand?" in plain English.

1.1 MOTIVATION

The motivation for this project comes from the everyday problems of small retail stores. A store owner has to keep track of hundreds of products, each with its own price, tax

rate, supplier and rate of sale. When this is done on paper, errors are unavoidable: a sale is not recorded, a delivery is counted twice, damaged goods are thrown away without being written off. The owner discovers the difference only when the shelf is empty or during an annual stock count. Lost sales from out-of-stock items and money tied up in excess stock directly reduce the store's profit.

Commercial retail software exists, but much of it is either expensive, designed for large chains, or limited to billing without real inventory control. Most systems also leave the hardest decision, what to reorder and how much, entirely to the owner's judgement. Recent advances in time-series forecasting and in large language models make it possible to build a system that not only records data accurately but also helps the owner act on it, while leaving the final decision with the owner.

As engineering students of Artificial Intelligence and Machine Learning, we were motivated to build a complete, production-quality system rather than a prototype: one that handles the real rules of Indian retail (GST slabs, HSN codes, GSTIN validation, financial-year invoice numbering, Indian number formatting), keeps data consistent even when several staff use it at the same time, and applies AI responsibly, with a human confirming every order the AI proposes.

1.2 OBJECTIVES

1. To design and develop a web-based system that manages the product catalogue, categories, suppliers, stock, purchase orders and sales of a retail store in one place.
2. To keep stock accurate by recording every change as an entry in a stock ledger inside a database transaction, so that stock always equals the sum of its movements and can never become negative.
3. To produce GST-compliant purchase orders and invoices, with HSN codes, correct CGST/SGST or IGST and validated GSTINs.
4. To provide secure, role-based access for the admin (owner), managers and cashiers.
5. To forecast product demand from past sales and prepare restock plans that are placed as purchase orders only after the owner confirms them.
6. To let the owner query the business in plain English through an AI assistant that uses the system's own data.
7. To ensure quality through automated testing, continuous integration and realistic demonstration data.

1.3 SCOPE OF THE PROJECT

The system is intended for a single retail store with one or more billing counters and a small team of staff. It covers the full cycle of goods through the store: setting up products and

suppliers, ordering from suppliers, receiving deliveries (including partial deliveries), keeping stock accurate through counts and write-offs, selling to customers at the point of sale, handling returns, and reporting on sales, stock value and GST. It runs in a web browser on a desktop, laptop, tablet or mobile phone, so no special hardware is required; a USB barcode scanner, which behaves like a keyboard, can be used to add products quickly.

The scope includes three roles. The **admin** (owner) manages staff accounts and has full access. The **manager** runs the store: catalogue, suppliers, purchasing, stock adjustments, reports and the AI features. The **cashier** bills customers and processes returns, and cannot see cost prices or change stock directly.

The AI features are designed as decision support. The forecasting engine runs inside the system and needs no external service; the Gemini assistant is optional and is enabled only when an API key is configured. The AI never places an order on its own: confirming a restock plan is a separate action that only a signed-in person with purchasing rights can perform. The project does not cover online sales, multiple branches with stock transfers, or payroll and full accounting, which are noted as future enhancements.

1.4 APPLICATIONS

The system can be applied wherever goods are bought from suppliers and sold to customers in a physical store:

1. **General and grocery stores (kirana):** keep daily essentials such as milk, rice, oil and pulses in stock, with loose goods sold by weight.
2. **Supermarkets and mini-marts:** manage thousands of products across categories with barcode scanning at several counters.
3. **Stationery and book shops:** prepare for the sharp rise in demand when schools reopen.
4. **Electronics and accessories shops:** track higher-value items, costs and margins accurately.
5. **Dairy, bakery and beverage outlets:** reorder fast-moving and seasonal products before they run out.
6. **Pharmacies and cosmetic stores:** record expired and damaged stock with reasons for write-off.
7. **Wholesale distributors:** manage purchase orders, partial deliveries and supplier lead times.
8. **Training and education:** demonstrate inventory control, GST billing and demand forecasting to students of commerce and computer science.

CHAPTER 2

LITERATURE SURVEY

[1] "Development of a Web-Based Platform for Automating an Inventory Management of a Small and Medium Enterprise" was authored by F. F. Agboola, Y. M. Malgwi, M. A. Mahmud and J. P. Oguntoye, and was published in 2022 in the FUDMA Journal of Sciences. The paper observes that many businesses still keep stock records in spreadsheets, a practice the authors describe as labour-intensive, costly and error-prone. To address this, they develop computerised, web-based inventory management software that maintains stock records, tracks employee salaries, and updates sales and transaction data. The system is organised into modules so that it is user-friendly and adaptable, and it uses a MySQL database as its back end. A case study illustrates the approach, and the authors conclude that the software can replace manual stock recording and improve the effectiveness, efficiency and service delivery of the enterprise. The work does not include GST billing, a purchase-order workflow or demand forecasting.

[2] "An Inventory Management System for Small Businesses" was authored by H. Q. Lim, A. R. Rahmat and S. Suban@Ani, and was published in 2025 in the Journal of Digital System Development. The paper presents EzStock, a web-based inventory management system for small businesses that rely either on outdated manual methods or on software too complex for their needs. The system provides real-time visibility of stock levels and automates key inventory processes, and was built with HTML, CSS, JavaScript and PHP on a MySQL database. Requirements were gathered through a survey of 30 participants and a literature review, and development followed an Agile methodology. The evaluation reports an overall user satisfaction of 93.3%, with 90% of respondents finding the interface intuitive. The authors identify better search, reporting and product bundling as future work; forecasting and AI-assisted restocking are not addressed.

[3] "A Point-of-Sale Platform for the Retail Small and Medium Enterprises" was authored by P. Phuthaikampanatpinya and C. Dechsupa, and was published in 2023 in the Academic Journal of Science and Technology, Dhonburi Rajabhat University. The paper presents a web-based point-of-sale platform intended to give smaller retailers capabilities comparable to those of large retail operators, covering inventory management, promotions and sales. Requirements were derived from fresh food and beverage entrepreneurs and electrical appliance stores, and the framework was implemented with PHP and the MySQL database management system. Two entrepreneurs evaluated the platform and rated the overall result as good, and the authors conclude that it is practical and adaptable to other kinds of retail business. The platform does not address Indian GST invoicing, supplier purchase orders or demand forecasting.

[4] "Account + GST Inventory Management ERP System (Using Cloud Storage)" was authored by H. R. Patil, S. R. Kotgire, N. S. Madhekar, P. N. Shelke and I. Raskar, and was published in 2022 in the International Journal for Research in Applied Science and Engineering Technology (IJRASET). The paper addresses the accounting, Goods and Services Tax and inventory management needs of small businesses through an Enterprise Resource Planning (ERP) system that uses cloud storage. The authors note that ERP systems are valuable for integrating business functions but are expensive to deploy and maintain, so they have been adopted mainly by large organisations, leaving small and medium enterprises with limited access. They therefore propose a combined account, GST and inventory management system hosted on cloud storage as an affordable alternative. The work is close to the Indian context of this project but does not include demand forecasting, owner-confirmed restocking or a conversational assistant.

[5] "Applying Machine Learning in Retail Demand Prediction—A Comparison of Tree-Based Ensembles and Long Short-Term Memory-Based Deep Learning" was authored by M. Nasseri, T. Falatouri, P. Brandtner and F. Darbanian, and was published in 2023 in Applied Sciences (MDPI). The paper compares tree-based ensemble models with Long Short-Term Memory (LSTM) networks for retail demand prediction, using more than six years of daily demand data from a retailer covering over 330 products and about 5.2 million records, together with meteorological and COVID-19-related variables. Focusing on perishable product categories, the study compares the Extra Trees Regressor with other ensembles including XGBoost and with LSTM. The results show that the Extra Trees Regressor outperforms LSTM on MAPE, MAE, RMSE and R^2, with the largest advantage for fresh meat and only a marginal one for fruit. The study indicates that simpler models can match or exceed deep learning for practical retail forecasting, but the forecasts are not integrated into an operational inventory system.

[6] "Enhancing Supply Chain Agility and Sustainability through Machine Learning: Optimization Techniques for Logistics and Inventory Management" was authored by V. Pasupuleti, B. Thuraka, C. S. Kodete and S. Malisetty, and was published in 2024 in Logistics (MDPI). The paper investigates machine-learning techniques for optimising logistics and inventory management with the aim of improving supply chain agility and sustainability. Using historical data from a multinational retail corporation covering sales, inventory levels, order fulfilment rates and operational costs, the authors apply regression, classification, clustering and time-series analysis to demand forecasting, inventory optimisation and customer segmentation. The reported results show better demand forecasting accuracy, fewer overstock and stock-out situations, and accurate prediction of order fulfilment timelines. The work targets a large retailer and does not involve the store owner in confirming replenishment decisions.

[7] "Deep Reinforcement Learning for Inventory Control: A Roadmap" was authored by R. N. Boute, J. Gijsbrechts, W. van Jaarsveld and N. Vanvuchelen, and was published in 2022 in the European Journal of Operational Research. This invited review examines how deep reinforcement learning (DRL) can be applied to inventory control. The authors note that DRL has shown considerable promise for sequential decision-making and that early studies have applied it to inventory replenishment, but that a DRL algorithm involves many design choices whose tuning is computationally expensive, which makes practical use difficult. To help practitioners, they catalogue the key design decisions in implementing DRL for inventory control and discuss their implications, and they outline research directions that build on structural results from classical inventory theory. The roadmap assumes fully automated policies, whereas the proposed system keeps the owner in control of every order.

[8] "A Survey on Employing Large Language Models for Text-to-SQL Tasks" was authored by L. Shi, Z. Tang, N. Zhang, X. Zhang and Z. Yang, and was published in 2025 in ACM Computing Surveys. The paper surveys methods that use large language models (LLMs) to translate natural-language questions into SQL queries over relational databases. The authors observe that the development of LLMs has produced a large range of new Text-to-SQL methods. The survey reviews the benchmark datasets and evaluation metrics used in the field, examines the two mainstream approaches of prompt engineering and fine-tuning with a taxonomy and practical insights for each, and discusses the base models used. It is a useful reference for designing natural-language interfaces to business data, although it is not applied to retail inventory; the proposed assistant instead calls the system's own services as tools rather than generating database queries.

[9] "SiriusBI: A Comprehensive LLM-Powered Solution for Data Analytics in Business Intelligence" was authored by J. Jiang et al., and was published in 2025 in the Proceedings of the VLDB Endowment. The paper presents SiriusBI, an LLM-powered business intelligence system built for practical deployment in industry. The authors identify three limitations of existing BI solutions: outdated legacy functionality, single-round SQL generation that cannot clarify the user's intent over several turns, and the high cost of adapting methods across domains. They propose an end-to-end architecture, a multi-round dialogue mechanism for clarifying queries and a data-conditioned strategy for choosing how SQL is generated. Deployed on Tencent's data platform, the system achieves over 93% accuracy in SQL generation and reduces analysts' query time from minutes to seconds. It is designed for enterprise analysts and is not connected to day-to-day stock, billing or restocking operations.

[10] "Artificial Intelligence in Retail – A Systematic Literature Review" was authored by C. Heins, and was published in 2023 in Foresight (Emerald). The paper presents a systematic literature review of AI research in retail, covering peer-reviewed English-language articles published between 2005 and 2021, analysed by research topic, theoretical characterisation

and the AI methods used. The findings show a marked increase in AI-related retail literature in recent years, but the field is described as fragmented in scope and limited in methodology. Eight research topics are identified, of which optimising the retail value chain and meeting customer expectations with AI emerge as the key areas. The review predates LLM-based assistants and does not address small Indian retailers.

2.1 COMPARISON TABLE

Table 2.1: Comparison of Existing Approaches and the Proposed System

Aspect	Existing / Manual Systems	Proposed System
Stock records	Paper registers or spreadsheets updated by hand; stock drifts from reality.	**Stock ledger**: every change recorded in a database transaction; stock always equals the sum of its movements.
Billing and tax	Hand-written bills or stand-alone billing machines not linked to stock.	**GST-compliant** invoices with HSN codes and automatic CGST + SGST or IGST.
Purchasing	Orders placed by phone after looking at shelves; deliveries not matched to orders.	**Purchase orders** with partial deliveries, overdue tracking and weighted-average cost.
Reorder decisions	Owner's guesswork or a fixed reorder level.	**AI demand forecasting** with seasonality and festivals; restock plans with safety stock.
Control over AI	Fully manual, or fully automatic reordering in large-enterprise systems.	**Human in the loop**: the owner reviews and confirms every AI restock plan.
Querying data	Fixed reports, or specialist BI tools for analysts.	**Plain-English assistant** (Google Gemini) that uses the shop's own data.
Access and security	Shared logins or none.	**Role-based access** with JWT, rotating refresh tokens and audit log.
Platform	Desktop software tied to one computer.	**Web application** usable on any device; one-command Docker deployment.

2.2 KEY FINDINGS

The survey shows that web-based inventory systems for small businesses [1], [2], [4] and point-of-sale platforms for retail SMEs [3] successfully replace manual records, but they treat stock as a number that is edited directly and generally stop at recording and reporting. Few of them combine purchasing, billing and stock control in one consistent transactional

model, and only some address the Indian GST requirements of HSN codes, GSTIN validation and the CGST/SGST/IGST split.

Research on demand forecasting [5], [6] shows that machine-learning and statistical models can predict retail demand accurately and reduce stock-outs and overstock, but these studies are usually carried out on large retailers' data and are not integrated into the day-to-day system a small shop actually uses. Work on reinforcement learning for inventory control [7] aims at fully automatic replenishment policies, which a small store owner is unlikely to trust with real money.

Large language models now make natural-language access to databases practical [8], [9], but existing solutions target enterprise analysts rather than shop owners and are not connected to operational actions such as preparing an order. Reviews of AI in retail [10] identify optimisation of the retail value chain as a key area while noting that the research remains fragmented.

Overall, there is a gap for an affordable, integrated system for small Indian retailers that keeps stock accurate through a transactional ledger, follows GST rules, and uses AI to prepare reorder decisions that the owner confirms, together with a conversational assistant over the shop's own data. The proposed system is designed to fill this gap.

CHAPTER 3

PROPOSED SYSTEM

3.1 PROBLEM STATEMENT

Small and medium retail stores lack an affordable, integrated system that keeps their stock records accurate, produces GST-compliant documents, and helps them decide what to reorder. Manual registers and disconnected billing tools lead to stock figures that do not match the shelves, lost sales from items that run out, money tied up in slow-moving goods, errors in tax calculation, and decisions based on guesswork. Existing software either stops at record-keeping or, at the enterprise level, automates reordering without the owner's control. There is therefore a need for a secure, web-based system that records every stock movement reliably, follows Indian GST rules, forecasts demand from past sales, and prepares restock orders that the owner reviews and confirms.

3.2 SYSTEM ARCHITECTURE

The system follows a three-tier client-server architecture, shown in Fig 3.1. The three tiers are kept in one code repository (a monorepo) with a shared package, so that the rules for money, GST and validation are written once and used by both the browser and the server.

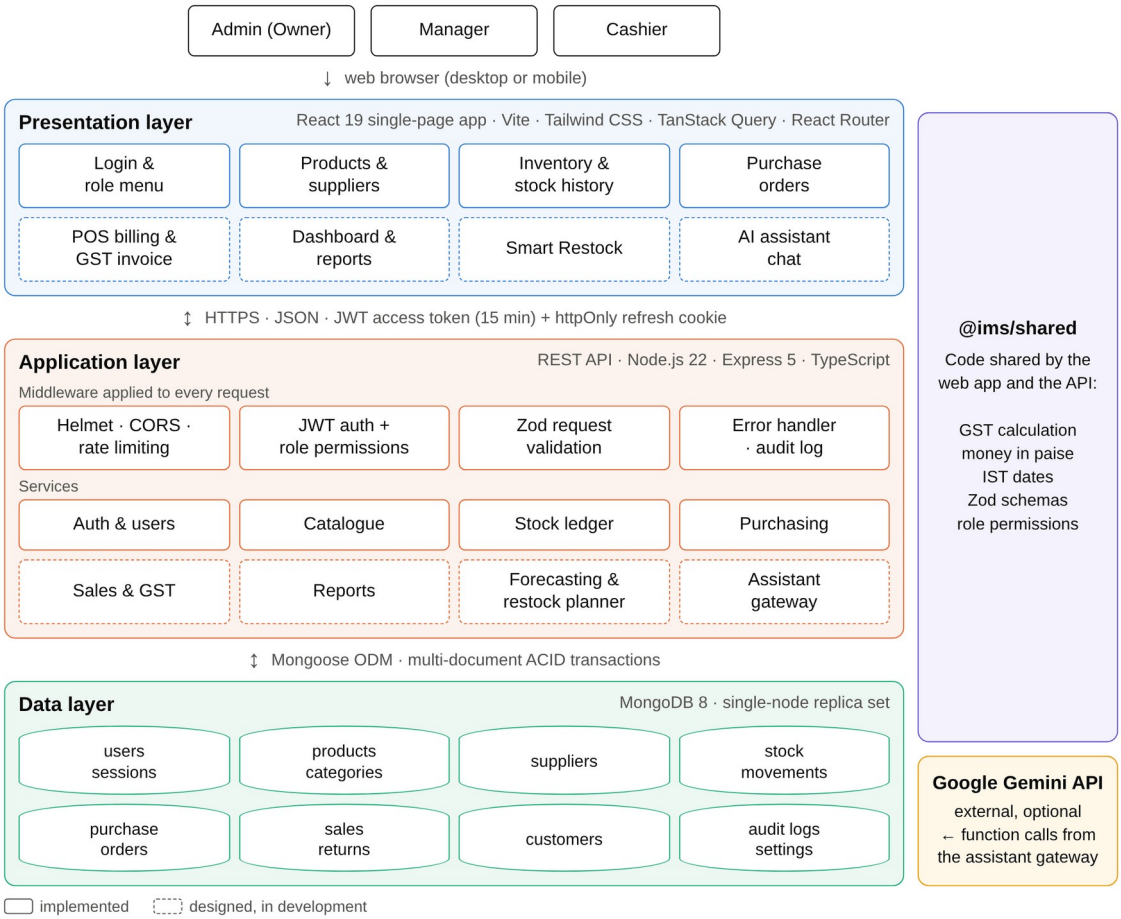


Fig 3.1: System Architecture

1. **Presentation layer (React single-page application):** runs in the browser and provides the screens for each module. It talks to the API over HTTPS with JSON. The short-lived access token is kept only in memory; when it expires, the app silently renews it using an httpOnly refresh cookie. Screens the user is not allowed to use are hidden according to the same permission list the server enforces.
2. **Application layer (Express REST API):** every request passes through security headers, CORS, rate limiting, JWT authentication, role-permission checks and Zod validation before reaching a service. Services contain the business logic: the catalogue, the stock ledger, purchasing, and (in development) sales with GST, reports, the forecasting and restock planner, and the assistant gateway. Important changes are written to an audit log.
3. **Data layer (MongoDB replica set):** stores the collections listed in Table 3.2. Running MongoDB as a replica set enables multi-document ACID transactions, so a sale, a purchase receipt or a stock adjustment either saves completely or not at all.
4. **Shared package (@ims/shared):** TypeScript code used by both tiers: GST calculation, money in paise, Indian financial-year dates and document numbers, request validation schemas and the role permission matrix.

5. **Google Gemini (optional external service):** used only by the assistant. The model is given tools that call the system's own services, so its answers are based on real figures rather than invented ones.

3.3 METHODOLOGY

The project was planned first and then built in ten iterative steps, one module at a time (Fig 3.2). Each step adds the database models, API endpoints and user interface for one module, together with automated tests. Before a step is considered complete, the module is checked in a real browser against realistic seed data, and the code is pushed to GitHub, where continuous integration runs formatting, linting, type checking, all tests, the production build and the Docker image build.

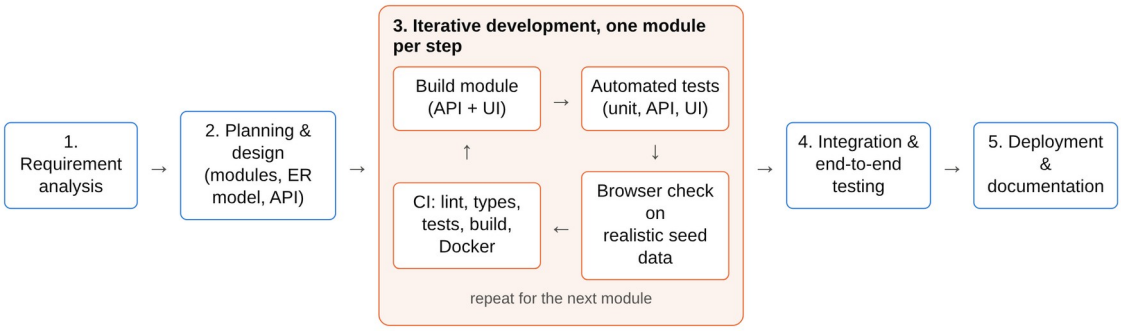


Fig 3.2: Development Methodology

The core of the AI restocking feature is the pipeline shown in Fig 3.3. It is divided into three stages: forecasting demand, deciding what to reorder, and the owner's decision.

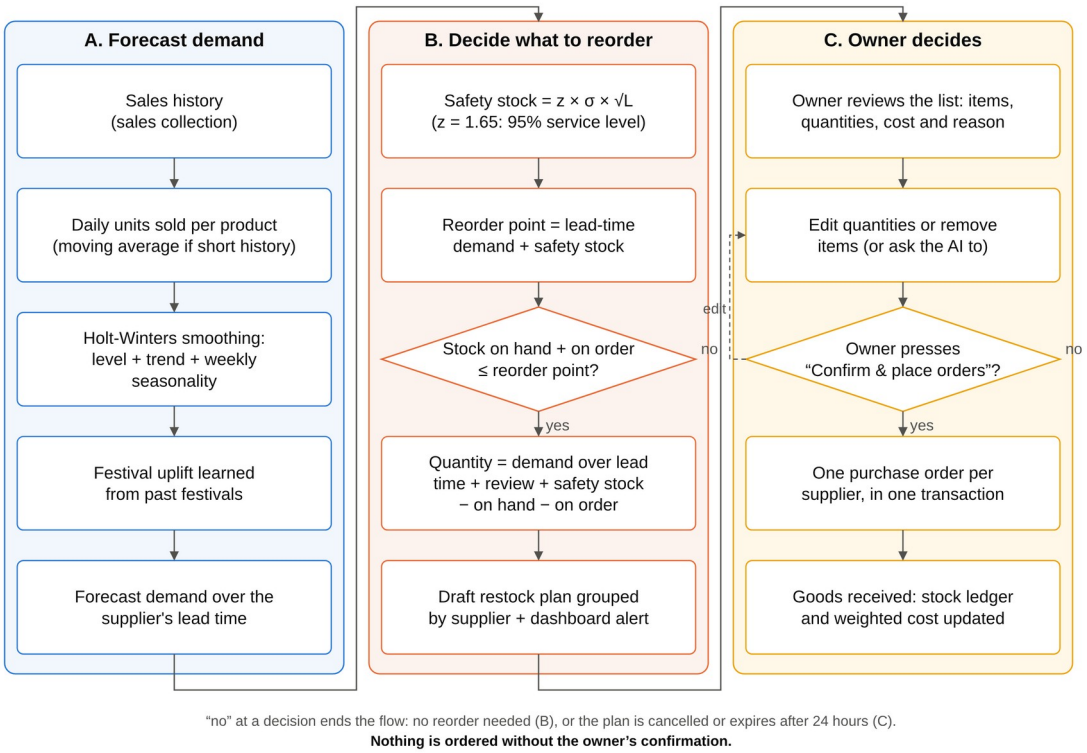


Fig 3.3: AI Restocking Methodology (Human in the Loop)

A. Forecasting demand

Daily units sold are aggregated for each product from the sales records, using Indian Standard Time calendar days. Demand is forecast with **Holt-Winters exponential smoothing** (triple exponential smoothing), which models three components: the level ℓ, the trend b and a weekly seasonal component s with period m = 7 days, because sales of a store differ strongly between weekdays and weekends (Fig 5.3). Daily sales of one product are noisy, so the trend is **damped** by a factor φ: a few slow or busy days cannot be projected weeks ahead. With smoothing constants α, β and γ and damping φ = 0.8, the additive form is:

$$\ell_t = \alpha\,(y_t - s_{t-m}) + (1-\alpha)(\ell_{t-1} + \varphi\, b_{t-1}) \quad (3.1)$$

$$b_t = \beta\,(\ell_t - \ell_{t-1}) + (1-\beta)\,\varphi\, b_{t-1} \quad (3.2)$$

$$s_t = \gamma\,(y_t - \ell_t) + (1-\gamma)\, s_{t-m} \quad (3.3)$$

$$\hat{y}_{t+h} = \ell_t + (\varphi + \varphi^2 + \ldots + \varphi^h)\, b_t + s_{t+h-m} \quad (3.4)$$

where:

- y_t is the number of units of the product actually sold on day t.
- ℓ_t (**level**) is the product's normal daily sales with the day-of-the-week effect taken out, as estimated on day t.
- b_t (**trend**) is how much the level is rising per day; it is negative when sales are falling.

- s_t (**seasonal effect**) is how far above or below the level sales usually are on that day of the week, for example +6 units on a Saturday and −4 units on a Monday.
- $m = 7$ is the length of the season in days (one week), so s_{t-m} is the seasonal effect of the same weekday one week earlier.
- h is the number of days ahead being forecast, and $\hat{y}_{t+h}$ is the forecast of the units that will be sold on day $t + h$.
- α, β and γ are smoothing constants between 0 and 1 for the level, the trend and the seasonal effect. A value close to 1 gives the newest day a lot of weight, so the estimate reacts quickly; a value close to 0 keeps the estimate steady.
- $\varphi = 0.8$ is the trend damping factor: each day further ahead, the trend counts for 0.8 of the day before, so the trend can add at most 4 days' worth of change to any forecast.

Equation (3.1) updates the level. The day's sales are first corrected for the day of the week by subtracting the usual effect of that weekday, $y_t - s_{t-m}$, so that a busy Saturday is not mistaken for a real rise in demand. This corrected value is then blended with what the previous level and damped trend expected for today, $\ell_{t-1} + \varphi\, b_{t-1}$: a share α comes from the new day and a share $1 - \alpha$ from the earlier estimate.

Equation (3.2) updates the trend. The change in level from yesterday to today, $\ell_t - \ell_{t-1}$, is blended with the previous (damped) trend. With a small β the trend changes only when sales keep rising or falling for some time, not because of one unusual day.

Equation (3.3) updates the seasonal effect of today's weekday. The difference between today's sales and the new level, $y_t - \ell_t$, is blended with the effect the same weekday had a week ago. Each of the seven weekdays keeps its own seasonal effect, refreshed once a week.

Equation (3.4) makes the forecast. It starts from the current level, adds the trend for each day ahead with less weight the further ahead it is ($\varphi + \varphi^2 + \ldots + \varphi^h$, which never exceeds $\varphi / (1 - \varphi) = 4$) and adds the seasonal effect of the weekday being forecast, taken from the most recent week (s_{t+h-m}). The demand expected during a supplier's lead time of L days is the sum of the forecasts for $h = 1$ to L.

The model is started from the first two weeks of sales: the starting level is the average daily sales of the first two weeks, the starting trend is the difference between the averages of the two weeks divided by seven, and each weekday's starting seasonal effect is its sales in the first two weeks minus those weeks' averages. After every update the seven seasonal effects are shifted so that they average zero, with the shift moved into the level, so that the weekly pattern cannot slowly absorb the level. For each product, α (0.02 to 0.7), β (0 to 0.05) and γ (0.02 to 0.3) are chosen by a grid search: every combination is run over the product's past sales and the one with the smallest squared error of its one-day-ahead forecasts is kept. Days

on which the shelf was empty at closing are treated specially: what sold that day is only a lower bound on what customers wanted, so the model uses the larger of the sales and its own forecast and leaves that day out of the error, instead of learning that nobody wants the product.

Worked example. A packet of biscuits normally sells about 20 units a day. Yesterday's estimates were a level $\ell_{t-1} = 20$ units and a trend $b_{t-1} = 0.5$ unit per day, and last Saturday's seasonal effect was $s_{t-7} = +6$ units. Today is a Saturday and 28 packets were sold. With $\alpha = 0.3$, $\beta = 0.1$, $\gamma = 0.2$ and $\varphi = 0.8$ (the small re-centring step is left out for clarity):

- Level (3.1): $\ell_t = 0.3 \times (28 - 6) + 0.7 \times (20 + 0.8 \times 0.5) = 6.6 + 14.28 =$ **20.88** units.
- Trend (3.2): $b_t = 0.1 \times (20.88 - 20) + 0.9 \times 0.8 \times 0.5 = 0.088 + 0.36 =$ **0.448** unit per day.
- Seasonal effect (3.3): $s_t = 0.2 \times (28 - 20.88) + 0.8 \times 6 = 1.424 + 4.8 =$ **6.22** units.
- Forecast for next Saturday (3.4), with $h = 7$ so that $s_{t+h-m} = s_t$: the damping sum is $0.8 + 0.8^2 + \ldots + 0.8^7 = 3.16$, so $\hat{y}_{t+7} = 20.88 + 3.16 \times 0.448 + 6.22 = 28.52$, that is about **29 packets**.

The system therefore expects about 29 packets to sell next Saturday: more than on an ordinary day because Saturdays are busy, and a little more than this Saturday because sales are growing. Without damping the trend would add $7 \times 0.448 = 3.1$ packets instead of 1.4, and over a month $30 \times 0.448 = 13.4$ packets instead of 1.8: a few busy days would be projected far into the future, which is why it is damped.

For products with less than four weeks of history, the average of recent days is used instead. Festivals such as Diwali, Ganesh Chaturthi and Eid cause short, sharp increases in demand for certain goods. The days of each festival's shopping build-up are left out of the fit; instead, the ratio of what sold to what the model expected is recorded, and a lift U for each product and festival is fitted by least squares so that sales at the peak of the build-up are U times a normal day, rising gradually over the build-up. Because one festival gives only a few days of evidence, each festival's lift is pulled towards the product's lift over all festivals, which in turn is pulled towards 1× (no change). The lift is applied to the known dates of upcoming festivals.

Forecast accuracy is measured on data the model has not seen: the model is fitted on all but the last four weeks, forecasts those four weeks, and the weekly totals are compared with what actually sold, using the Mean Absolute Percentage Error, $\text{MAPE} = (100/n)\ \Sigma\ |y_w - \hat{y}_w| / y_w$, over the n weeks (weeks with an empty shelf are left out, because their true demand is unknown). For example, a MAPE of 15% means the weekly forecasts were off by 15% on average. Because MAPE favours forecasts that are too low, the weighted error (total absolute error ÷ total sales) and the bias (forecast total ÷ actual total − 1) are reported too, and all three

are compared with a simple 28-day moving average. The Smart Restock page shows these figures for the shop's own data.

B. Deciding what to reorder

For each product the expected demand over the supplier's lead time L is computed from the forecast. Safety stock protects against demand being higher than forecast: safety stock = $z \times \sigma \times \sqrt{L}$, where σ is the standard deviation of daily forecast errors and $z = 1.65$ gives a 95% service level. The reorder point is the lead-time demand plus the safety stock. When the stock on hand plus the quantity already on order falls to or below the reorder point (or to the product's own reorder level, if the owner has set a higher one), or would not last until a delivery ordered today arrives, the product is added to a restock plan with a suggested quantity = demand over the lead time and a one-week review period + safety stock − stock on hand − quantity on order, rounded up to the pack size. A product with stock on the shelf and no sale for 60 days is dead stock and is never reordered. Each item carries a short reason, for example "sales up 40% this month; supplier takes 3 days".

C. The owner decides

Items are grouped by supplier into a draft restock plan, and the dashboard shows an alert. The owner sees the list of items, quantities, costs and reasons, can change quantities or remove items, and then confirms or cancels the plan. The owner may also set a budget, in which case the most urgent items first get one pack each, then enough to last until their delivery, and then the rest of their suggestion, so that a small budget covers many items a little rather than one item fully. Only on confirmation does the system create one purchase order per supplier, inside a single transaction. Confirmation first re-checks the goods on order: if any of the items were ordered since the plan was made, the plan expires instead, so nothing is ordered twice. A plan that is not confirmed within 24 hours, or that is replaced by a newer plan, also expires.

3.4 DATA FLOW

Fig 3.4 shows the context-level (level 0) data flow diagram. The system exchanges data with five external entities: the admin (owner), managers and cashiers who use it, customers who receive invoices, suppliers who receive purchase orders and deliver goods, and the optional Google Gemini service.

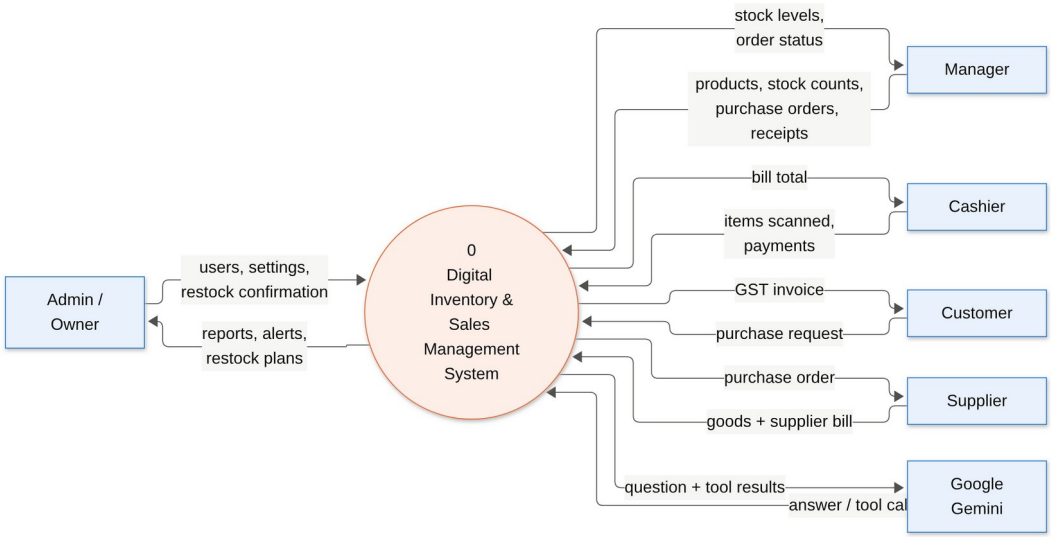


Fig 3.4: Data Flow Diagram – Level 0 (Context Diagram)

Fig 3.5 breaks the system into its main processes. The key design point is visible in the diagram: purchasing (4.0) and POS billing (5.0) never change stock themselves; they send the quantities to the stock movement process (3.0), which updates the product's stock and writes the ledger entry together. The forecasting process (6.0) reads the sales history and current stock, and its output reaches purchasing only as a plan confirmed by the owner.

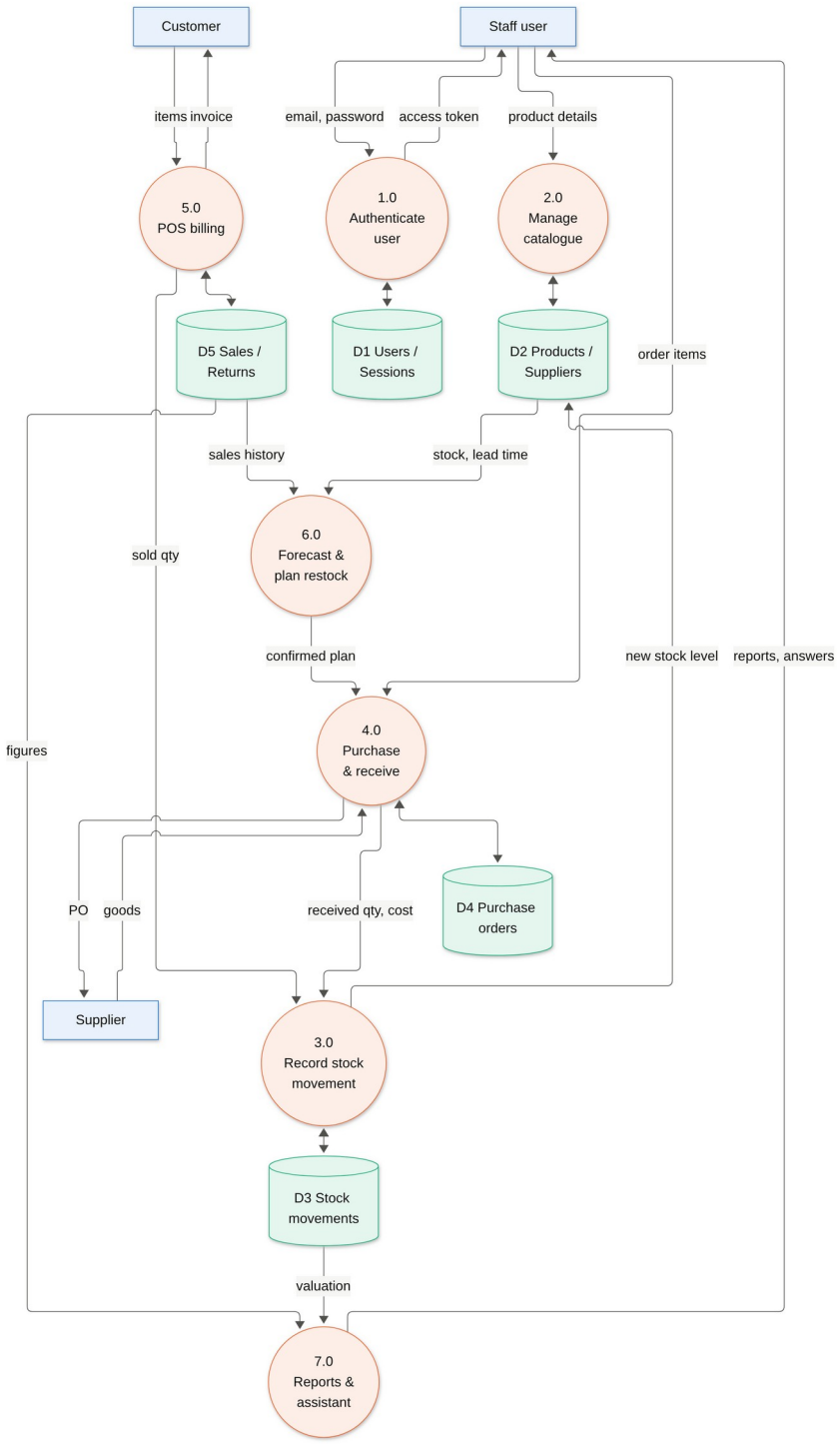


Fig 3.5: Data Flow Diagram – Level 1

The data flow is organised as follows:

1. **Frontend (client side):** React pages collect input through validated forms (React Hook Form with Zod), show live calculations such as GST and margin while the user types, and send JSON requests to the API. Data is cached and refreshed with TanStack Query; filters are kept in the page address so that a filtered list can be bookmarked or shared.

2. **Backend (server side):** the API authenticates the request, checks the role's permissions, validates the body, and calls a service. Services that change stock run inside a MongoDB transaction and call the stock ledger service, which performs one atomic update that succeeds only if enough stock remains.
3. **Database:** MongoDB stores the documents; indexes support searching products by name, SKU and barcode, listing stock movements by product and date, and aggregating sales by day.
4. **AI services:** the forecasting engine reads sales and stock through aggregation queries and writes draft restock plans; the assistant gateway sends the owner's question and tool results to Gemini and returns the answer.

3.5 USE CASE DIAGRAM

Fig 3.6 shows the use cases of the system for each role. The admin has every permission; the manager runs the store day to day; the cashier is limited to billing and returns. Table 3.1 lists the permissions that are enforced on every API request.

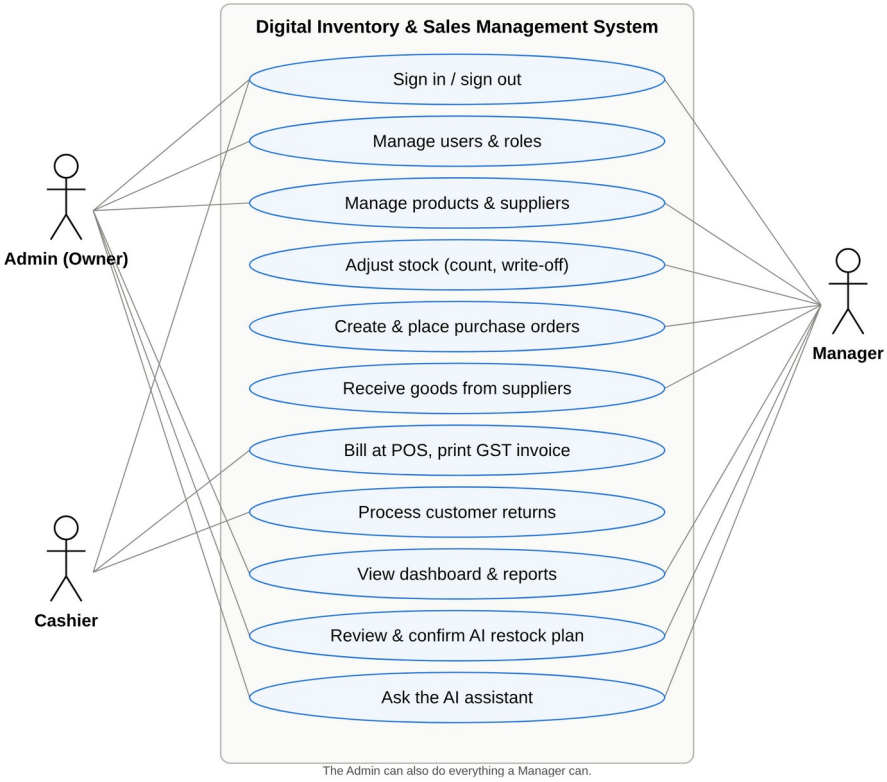


Fig 3.6: Use Case Diagram

Table 3.1: Role Permission Matrix

Capability	Admin	Manager	Cashier
Manage users and shop settings	✓		
Manage products, categories and suppliers	✓	✓	

Capability	Admin	Manager	Cashier
Adjust stock, create and receive purchase orders	✓	✓	
Bill customers (POS) and process returns	✓	✓	✓
View cost prices, stock value and reports	✓	✓	
Use forecasts, restock plans and the AI assistant	✓	✓	
View the audit log	✓		

3.6 DATABASE DESIGN

The database consists of thirteen collections (Table 3.2). Three design decisions are central. First, all amounts of money are stored as integers in paise (1 rupee = 100 paise), which avoids the rounding errors of floating-point numbers. Second, a product's stock is never edited directly: it changes only together with a stock movement that records the type, quantity, resulting balance, cost, user and source document. Third, sales and purchase orders store a snapshot of the product name, HSN code, price and tax at the time of the transaction, so later price changes never alter past invoices.

Table 3.2: Database Collections

Collection	Purpose
users	Staff accounts with name, email, bcrypt password hash, role and active status.
sessions	Refresh-token sessions (SHA-256 hash only), with rotation and reuse detection.
settings	Shop name, address, state, GSTIN and document number prefixes.
counters	Gap-free sequence numbers per financial year for invoices, POs and returns.
categories	Product groups such as Dairy, Beverages and Staples.
suppliers	Supplier details, GSTIN (validated against the state), lead time in days.
products	SKU, barcode, HSN, GST rate, cost and selling price, MRP, unit, stock, reorder level.
customers	Optional customer details; GSTIN and state decide CGST + SGST or IGST.
purchaseorders	Orders to suppliers with items, GST, status and the history of deliveries.
sales	Invoices with items, discounts, CGST/SGST/IGST, round-off and payments.
salereturns	Returns linked to the original invoice and its lines.

Collection	Purpose
stockmovements	The stock ledger: one entry for every change in stock.
auditlogs	Who changed what and when (e.g. price changes, orders placed, users deactivated).

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 SOFTWARE REQUIREMENTS

1. **Operating System:** Windows 10/11, Linux (Ubuntu 22.04 or later) or macOS for development. The application runs in Docker containers, so the same setup works on any of them; users only need a modern web browser (Chrome, Edge, Firefox or Safari).
2. **Programming Language:** TypeScript 6 (strict mode) for the frontend, backend and shared code, running on Node.js 22.
3. **Frontend:** React 19, Vite 8 (build tool), Tailwind CSS 4 (styling), React Router (navigation), TanStack Query (server data), React Hook Form with Zod (forms), Lucide icons.
4. **Backend:** Express 5, Mongoose 9 (MongoDB object modelling), Zod 4 (validation), jose (JSON Web Tokens), bcryptjs (password hashing), Helmet and express-rate-limit (security), Pino (logging).
5. **Database:** MongoDB 8 running as a single-node replica set to support transactions.
6. **AI:** forecasting engine implemented in TypeScript within the API (no external service); Google Gemini API through a Google AI Studio key for the assistant.
7. **Testing and quality:** Vitest, Supertest, Testing Library, mongodb-memory-server, Playwright, ESLint, Prettier.
8. **Tools:** Visual Studio Code, Git and GitHub, GitHub Actions (continuous integration), Docker and Docker Compose, MongoDB Compass.

Table 4.1: Technology Stack

Layer	Technology	Role in the system
Frontend	React 19, Vite 8, Tailwind CSS 4	Responsive single-page user interface, light and dark mode
Backend	Node.js 22, Express 5, TypeScript	REST API, business logic, security
Database	MongoDB 8, Mongoose 9	Document storage, ACID transactions, aggregations
Validation	Zod 4 (shared)	Same rules in the browser and on the server
Security	JWT (jose), bcrypt, Helmet	Authentication, password storage, HTTP headers
AI	Holt-Winters engine, Google Gemini	Demand forecasting, restock plans, assistant

Layer	Technology	Role in the system
DevOps	Docker Compose, GitHub Actions	One-command setup, automated checks on every push

4.2 HARDWARE REQUIREMENTS

1. **Processor:** Intel Core i3 or equivalent (minimum); Intel Core i5 or equivalent recommended for development with Docker.
2. **RAM:** 4 GB minimum for running the application; 8 GB or more recommended for development.
3. **Storage:** 10 GB of free space for the tools, Docker images and database; an SSD is recommended.
4. **Network:** a local network for the shop's devices; internet access only for the optional Gemini assistant.
5. **Client devices:** any desktop, laptop, tablet or smartphone with a modern browser.
6. **Optional peripherals:** USB or Bluetooth barcode scanner (works as a keyboard) and a printer for invoices.

4.3 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Functional requirements

1. The system shall allow staff to sign in and shall restrict each screen and action according to the user's role.
2. The system shall let managers add, edit, search and deactivate products, categories and suppliers, validating SKU, barcode, HSN code, GST rate, MRP and GSTIN.
3. The system shall record every change of stock as a stock movement and shall refuse any change that would make stock negative.
4. The system shall support stock counts and write-offs (damaged, expired, lost) with a reason.
5. The system shall create, place, receive (fully or partly) and cancel purchase orders, calculating GST per line.
6. The system shall bill customers at the point of sale with barcode search, discounts, GST and split payments, and print a GST invoice.
7. The system shall forecast demand, prepare restock plans and place purchase orders only after the owner confirms.
8. The system shall answer the owner's questions in plain English using the shop's data.

Non-functional requirements

1. **Consistency:** operations that change several records must be atomic; concurrent sales must never oversell stock.
2. **Security:** passwords hashed with bcrypt (cost 12); access tokens valid for 15 minutes; refresh tokens rotated on every use and revoked if reused; login rate-limited.
3. **Usability:** responsive layout for desktop and mobile, keyboard and barcode-scanner friendly, clear error messages next to the field concerned.
4. **Performance:** list pages paginated and indexed; typical API responses well under one second on the demonstration data.
5. **Maintainability:** strict TypeScript, shared validation, automated tests and continuous integration on every change.

4.4 DATASETS

A demand-forecasting system needs a history of sales to learn from, and a realistic dataset is also needed to demonstrate and test the application. Since the transaction records of a real shop could not be obtained or published, the project includes a **deterministic data generator** that simulates one full year of trading of a generic Indian retail store. Running it with the same seed always produces exactly the same data, so tests and demonstrations are repeatable.

The simulated store sells 44 products in eight categories bought from six suppliers in Karnataka, Tamil Nadu and Maharashtra. The generator models customer footfall day by day with the patterns a real store shows: busier weekends, higher spending in the week after salaries are paid, seasonal products (beverages and water in summer, mosquito repellents in the monsoon, notebooks when schools reopen in June, tea and coffee in winter), growing and declining products, and festival peaks around Diwali, Dussehra, Christmas, Makar Sankranti, Holi, Eid, Raksha Bandhan and Ganesh Chaturthi. Purchases are generated by the same reorder rules a shopkeeper would follow, deliveries arrive after each supplier's lead time, and the data also contains returns, stock counts, write-offs, a few slow-moving "dead stock" items and items deliberately left low, to demonstrate alerts.

Every invoice is internally consistent (its lines, taxes, round-off and payments add up), every product's stock equals the sum of its stock movements and is never negative, and every supplier GSTIN is valid. These properties are verified by automated tests. Table 5.1 summarises the generated dataset.

4.5 SAMPLE CODE

This section presents excerpts from the actual source code of the project. Fig 4.1 shows how the Express application is set up: security headers (Helmet), cross-origin rules

(CORS), JSON body parsing with a size limit, cookie parsing for the refresh token, request logging, the API routes and central error handling.

apps/api/src/app.ts

```ts
import cookieParser from 'cookie-parser';
import cors from 'cors';
import express from 'express';
import helmet from 'helmet';
import { pinoHttp } from 'pino-http';
import { env } from './config/env.js';
import { errorHandler, notFoundHandler } from './middleware/error-handler.js';
import { createApiRouter } from './routes/index.js';
import { logger } from './utils/logger.js';

export function createApp() {
  const app = express();

  app.disable('x-powered-by');
  app.set('trust proxy', 1);

  app.use(helmet());
  app.use(cors({ origin: env.CORS_ORIGIN, credentials: true }));
  app.use(express.json({ limit: '1mb' }));
  app.use(cookieParser());
  app.use(pinoHttp({ logger, autoLogging: env.NODE_ENV !== 'test' }));

  app.use('/api', createApiRouter());

  app.use(notFoundHandler);
  app.use(errorHandler);

  return app;
```

Fig 4.1: Express Application Setup

Fig 4.2 shows the Mongoose schema of a product. Validation rules are declared on the schema itself: the SKU format, an optional 8- to 14-digit barcode, the HSN code, the GST rate, and prices in paise. The stock field has a minimum of zero as a final safeguard against negative stock.

apps/api/src/models/product.ts

```
const productSchema = new Schema(
  {
    sku: {
      type: String,
      required: true,
      trim: true,
      uppercase: true,
      unique: true,
      match: [/^[A-Z0-9-]{3,32}$/, 'SKU may contain only letters, digits and dashes (3-32)'],
    },
    /** EAN-13 / UPC / internal barcode. Optional but unique when present. */
    barcode: { type: String, trim: true, match: [/^[0-9]{8,14}$/, 'Barcode must be 8-14 digits'] },
    name: { type: String, required: true, trim: true, maxlength: 120 },
    description: { type: String, trim: true, maxlength: 1000 },
    category: { type: Schema.Types.ObjectId, ref: 'Category', required: true },
    brand: { type: String, trim: true, maxlength: 60 },
    unit: { type: String, enum: UNITS, required: true, default: 'pcs' },
    /** Harmonised System of Nomenclature code printed on GST invoices (4, 6 or 8 digits). */
    hsnCode: {
      type: String,
      required: true,
      match: [/^([0-9]{4}|[0-9]{6}|[0-9]{8})$/, 'Invalid HSN code'],
    },
    gstRate,
    /** Purchase cost per unit, excluding GST (input tax is claimed back). */
    costPrice: paise,
    /** Selling price per unit; includes GST when `taxInclusive` is true. */
    sellingPrice: paise,
    /** Maximum retail price printed on the pack, if any. */
    mrp: { ...paise, required: false },
    taxInclusive: { type: Boolean, default: true },
    /** Current stock on hand. Only changed together with a StockMovement. */
    stock: { type: Number, required: true, default: 0, min: [0, 'Stock cannot go negative'] },
    /** Raise a low-stock alert when stock falls to or below this level. */
    reorderLevel: { type: Number, required: true, min: 0, default: 10 },
    /** Default quantity to order when restocking. */
    reorderQuantity: { type: Number, required: true, min: 0, default: 50 },
    preferredSupplier: { type: Schema.Types.ObjectId, ref: 'Supplier' },
    imageUrl: { type: String, trim: true },
    isActive: { type: Boolean, default: true },
  },
  baseOptions,
```

Fig 4.2: Product Schema (MongoDB / Mongoose)

Fig 4.3 shows the GST calculation shared by the frontend and backend. For a supply within the state, the tax is split equally into CGST and SGST; across states, IGST is charged at the full rate. Working in integer paise and rounding each half separately guarantees that CGST and SGST are always equal and that the totals add up exactly.

packages/shared/src/gst.ts

```ts
/**
 * Splits tax on a taxable amount.
 * Intra-state supply (seller and buyer in the same state) -> CGST + SGST, half the rate each.
 * Inter-state supply -> IGST at the full rate.
 */
export function calculateGst(taxable: Paise, rate: GstRate, interState: boolean): GstBreakup {
  if (!Number.isInteger(taxable) || taxable < 0) {
    throw new RangeError(`Taxable amount must be a non-negative integer of paise: ${taxable}`);
  }
  if (interState) {
    const igst = Math.round((taxable * rate) / 100);
    return { taxable, cgst: 0, sgst: 0, igst, totalTax: igst, total: taxable + igst };
  }
  const half = Math.round((taxable * rate) / 200);
  return {
    taxable,
    cgst: half,
    sgst: half,
    igst: 0,
    totalTax: half * 2,
    total: taxable + half * 2,
  };
}
```

Fig 4.3: GST Calculation (CGST + SGST or IGST)

Fig 4.4 shows the heart of the stock ledger. A single atomic database update adds the change to the stock only if enough stock remains for a removal; if two cashiers try to sell the last units at the same moment, only one succeeds. The movement is then recorded in the same transaction.

apps/api/src/modules/inventory/ledger.ts

```ts
export async function recordMovement(
  input: MovementInput,
  session: ClientSession,
): Promise<MovementResult> {
  const quantity = round3(input.quantity);
  if (quantity === 0) throw new Error('A stock movement cannot be zero');

  // One atomic update: add the change, but only if enough stock remains for a removal.
  // Rounding in the database keeps loose goods exact (2.5 kg − 0.3 kg = 2.2 kg, not 2.1999…).
  const product = await ProductModel.findOneAndUpdate(
    { _id: input.product, ...(quantity < 0 && { stock: { $gte: -quantity } }) },
    [{ $set: { stock: { $round: [{ $add: ['$stock', quantity] }, 3] } } }],
    {
      session,
      returnDocument: 'after',
      updatePipeline: true,
      timestamps: false,
      projection: { name: 1, stock: 1, costPrice: 1 },
    },
  ).lean();
```

Fig 4.4: Stock Ledger – Atomic Stock Update

Fig 4.5 shows how receiving goods updates the cost price of a product to the weighted average of the stock already on the shelf and the new delivery, so that stock valuation and profit reflect what the goods actually cost.

apps/api/src/modules/purchase-orders/service.ts

```
      const before = await ProductModel.findById(
        line.product,
        { stock: 1, costPrice: 1 },
        { session },
      ).lean();
      if (!before) throw fieldError(['items', index, 'product'], `${line.name} no longer exists`);
      await recordMovement(
        {
          product: line.product,
          type: 'purchase',
          quantity: received.quantity,
          unitCost: line.unitCost,
          ref: { model: 'PurchaseOrder', id: po._id },
          userId,
          occurredAt: now,
        },
        session,
      );
      // Weighted average cost: what the shelf as a whole cost to buy.
      const stockBefore = Math.max(0, before.stock);
      const costPrice =
        stockBefore > 0
          ? Math.round(
              (stockBefore * before.costPrice + received.quantity * line.unitCost) /
                (stockBefore + received.quantity),
            )
          : line.unitCost;
      await ProductModel.updateOne({ _id: line.product }, { $set: { costPrice } }, { session });
```

Fig 4.5: Weighted-Average Cost on Goods Receipt

Fig 4.6 and Fig 4.7 show the authentication middleware and the permission matrix. The user is re-loaded on every request, so deactivating an account or changing a role takes effect immediately. The same permission list is used by the web app to hide screens a user cannot use.

apps/api/src/middleware/auth.ts

```
 * Requires a valid `Authorization: Bearer <access token>` header.
 *
 * The user is re-loaded on every request, so deactivating an account or changing a role
 * takes effect immediately rather than when the token expires.
 */
export const authenticate: RequestHandler = async (req, _res, next) => {
  const header = req.get('authorization');
  if (!header?.startsWith('Bearer ')) throw AppError.unauthorized();

  const { userId } = await verifyAccessToken(header.slice('Bearer '.length).trim());
  const user = isValidObjectId(userId) ? await UserModel.findById(userId).lean() : null;
  if (!user || !user.isActive) {
    throw new AppError(401, 'Your account is no longer active', 'ACCOUNT_INACTIVE');
  }

  req.user = { id: String(user._id), name: user.name, email: user.email, role: user.role };
  next();
};

/** Allows the request only if the signed-in user's role has every listed permission. */
export function requirePermission(...permissions: Permission[]): RequestHandler {
  return (req, _res, next) => {
    if (!req.user) throw AppError.unauthorized();
    if (!permissions.every((p) => can(req.user!.role, p))) throw AppError.forbidden();
    next();
  };
}
```

Fig 4.6: Authentication and Permission Middleware

packages/shared/src/permissions.ts

```ts
  (p) => !['users:manage', 'settings:manage', 'audit:read'].includes(p),
);

const CASHIER: readonly Permission[] = [
  'products:read',
  'inventory:read',
  'sales:create',
  'sales:read',
  'returns:create',
  'customers:read',
  'customers:write',
];

export const ROLE_PERMISSIONS: Readonly<Record<Role, ReadonlySet<Permission>>> = {
  admin: new Set(PERMISSIONS),
  manager: new Set(MANAGER),
  cashier: new Set(CASHIER),
};

export function can(role: Role | undefined, permission: Permission): boolean {
  return role !== undefined && ROLE_PERMISSIONS[role].has(permission);
}
```

Fig 4.7: Role Permission Matrix in Code

CHAPTER 5

DATA VISUALIZATION

Before building the forecasting engine, the sales history was analysed to understand which patterns a forecast must capture. All figures in this chapter are produced from the one-year demonstration dataset described in Section 4.4 by querying the project's MongoDB database with aggregation pipelines and plotting the results with Matplotlib. Because the dataset is simulated, the charts illustrate the behaviour the system is designed to handle; they are not measurements of a real store.

Fig 5.1 shows the monthly sales revenue over eleven complete months. Revenue is fairly stable at ₹7–8 lakh per month, with the highest values in the festive and winter months and a dip in the late summer, when fewer festivals fall.

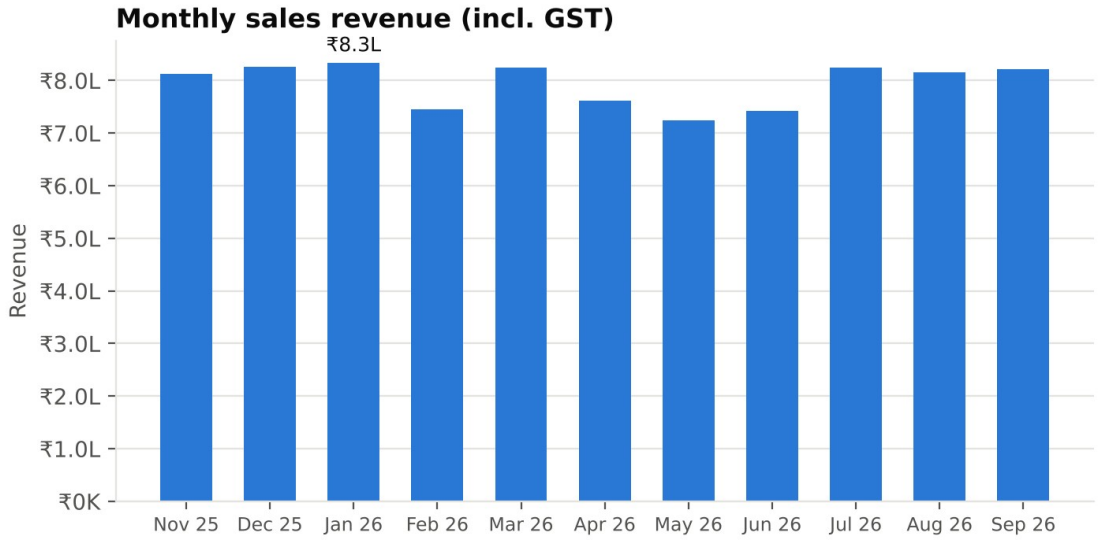


Fig 5.1: Monthly Sales Revenue

Fig 5.2 shows daily sales over the year with the main festivals marked, together with a seven-day moving average. Individual days vary a great deal, which is why a forecast cannot simply copy yesterday's sales. Peaks appear just before festivals such as Diwali, Holi, Eid and Ganesh Chaturthi, when festive goods sell several times faster than usual.

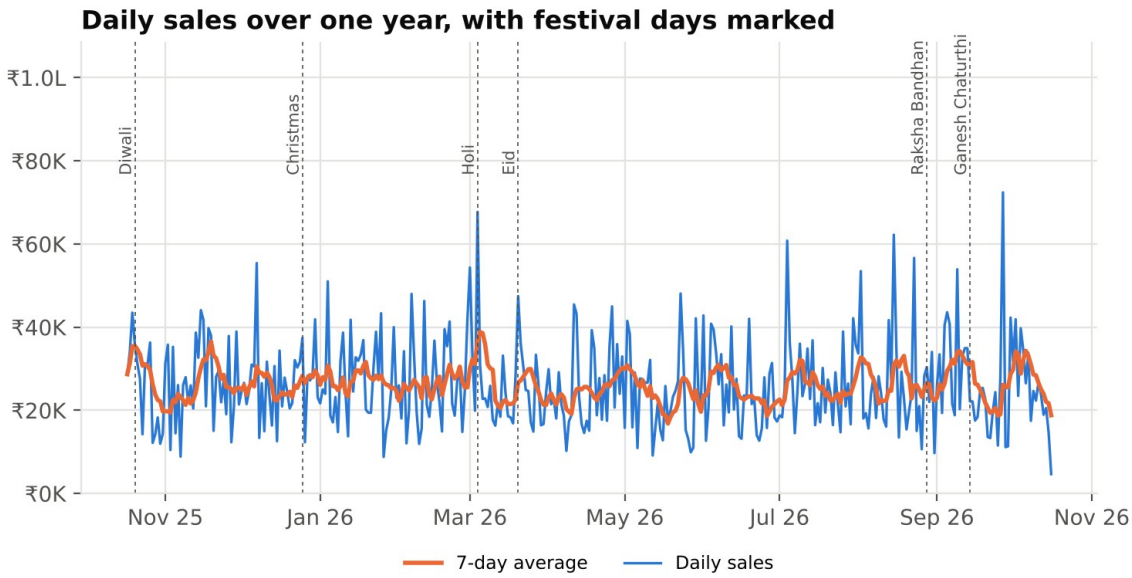


Fig 5.2: Daily Sales with Festival Days Marked

Fig 5.3 shows the average sales for each day of the week. Sales rise steadily from Monday to Sunday, and a Sunday brings in about 60% more than a Monday. This strong weekly cycle is the reason the forecasting engine uses a seasonal model with a seven-day period.

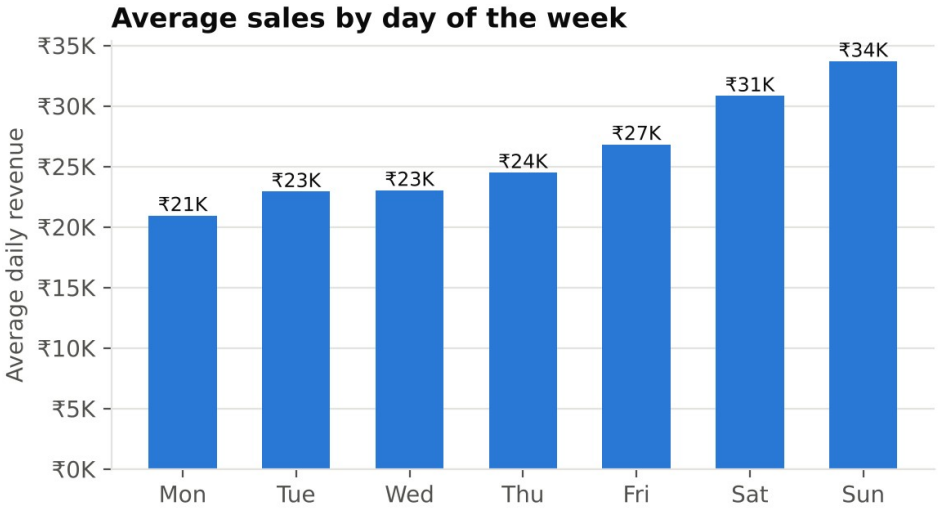


Fig 5.3: Average Sales by Day of the Week

Fig 5.4 compares the weekly demand of four products with different behaviours: toned milk sells steadily all year, drinking water peaks in summer, mosquito repellent peaks in the monsoon, and notebooks peak when schools reopen in June. A single fixed reorder level would be too high for one part of the year and too low for another; this is the main argument for forecasting each product separately.

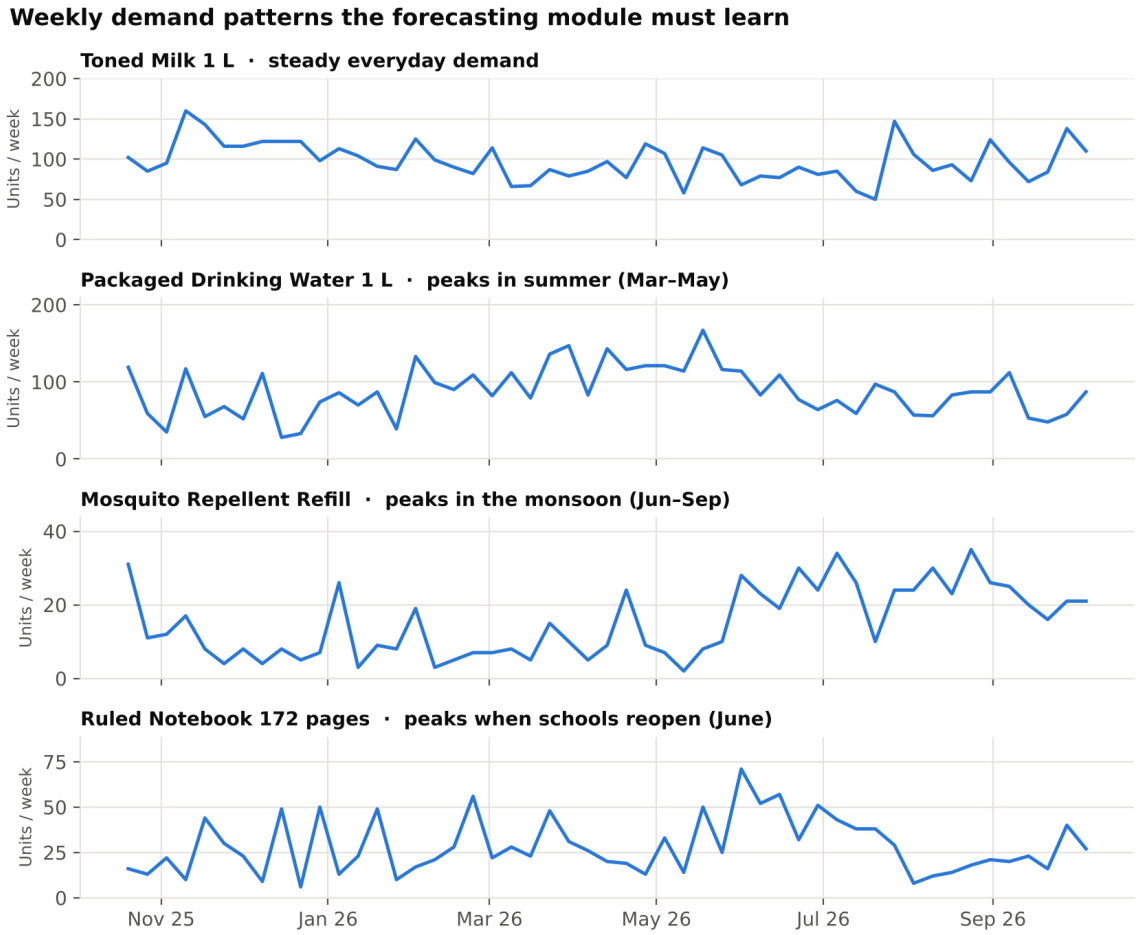


Fig 5.4: Weekly Demand Patterns of Four Products

Fig 5.5 and Fig 5.6 show where the revenue comes from. Staples and grains bring in 39% of revenue, followed by beverages and dairy. The top ten products are dominated by high-value staples such as rice, wheat flour, tea and oil, which are the items where a stock-out costs the most.

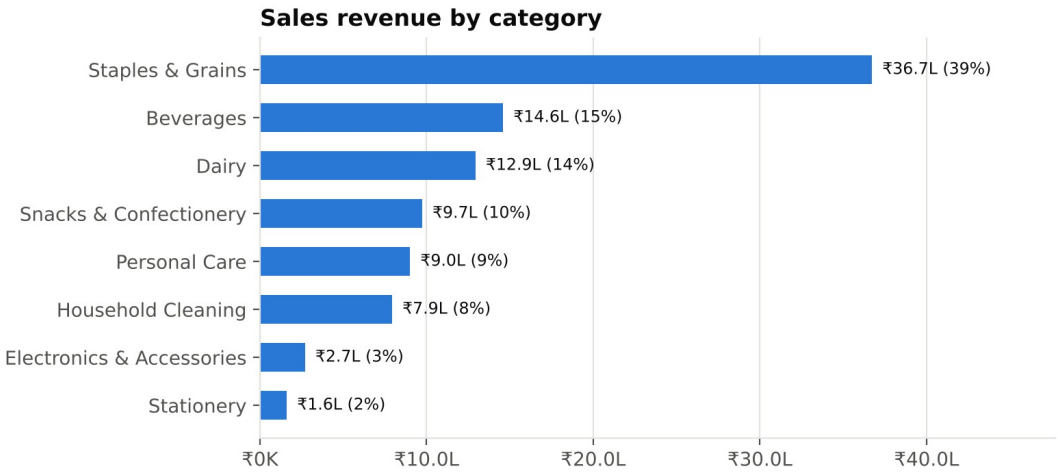


Fig 5.5: Sales Revenue by Category

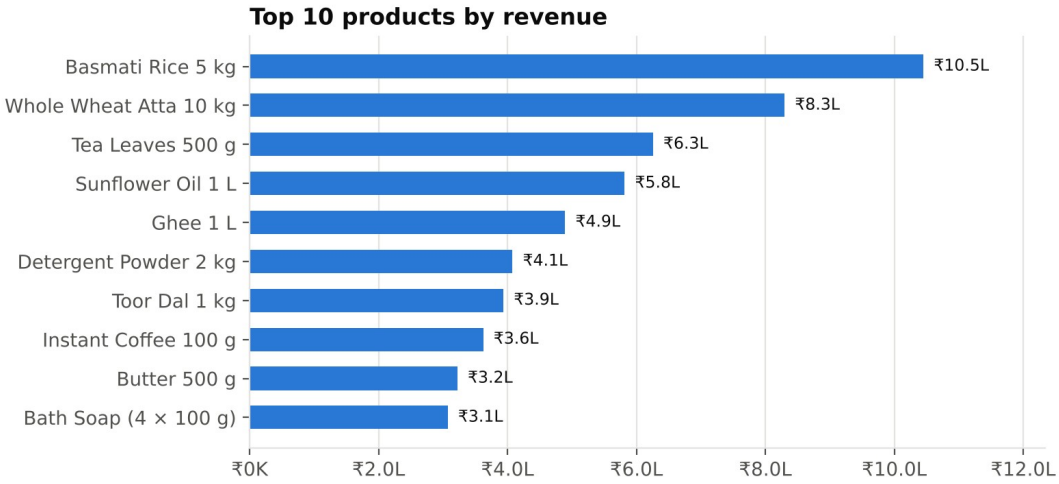


Fig 5.6: Top 10 Products by Revenue

Fig 5.7 shows the share of sales value by payment method: UPI accounts for nearly half of sales, reflecting how digital payments have become common in Indian retail. The point-of-sale module therefore supports UPI, cash and card, including a bill split across methods. Fig 5.8 shows the current stock status of the 44 active products: 13 are at or below their reorder level and 3 are out of stock, exactly the situation the restocking agent is designed to catch early.

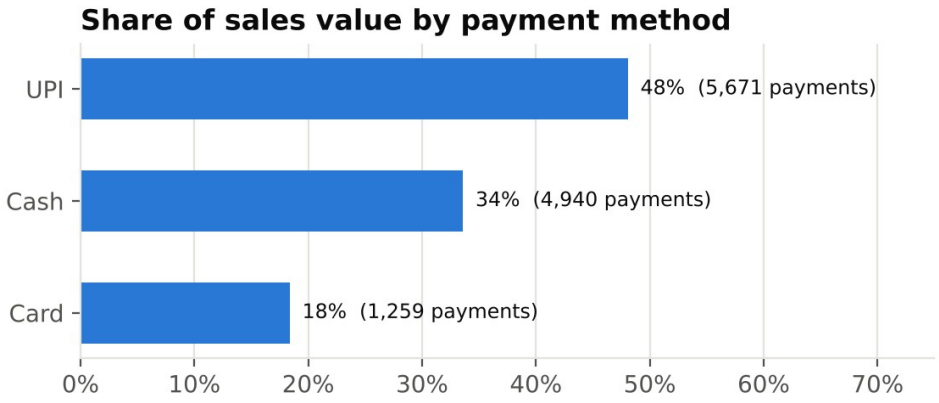


Fig 5.7: Share of Sales Value by Payment Method

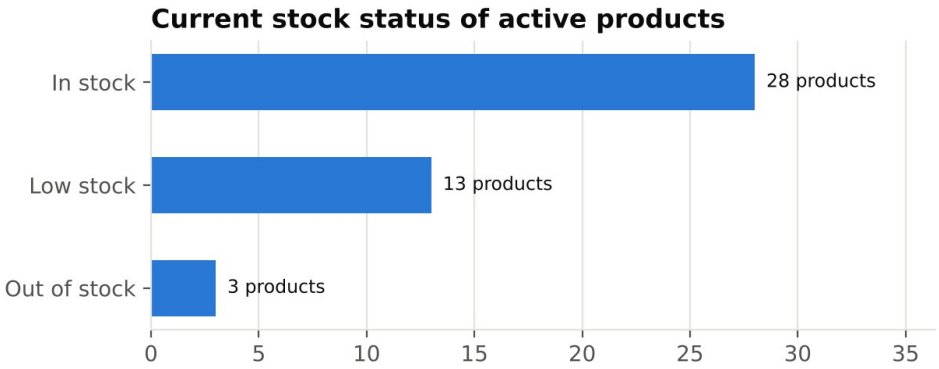


Fig 5.8: Current Stock Status of Active Products

Table 5.1 summarises the demonstration dataset.

Table 5.1: Summary of the Demonstration Dataset

Item	Count	Notes
Products / categories / suppliers	44 / 8 / 6	Suppliers in 3 states
Customers	84	Some with GSTIN
Sales invoices	11,870	One year, IST days
Total sales value	₹95,24,504	Including GST
Purchase orders	796	Incl. partial deliveries
Customer returns	95	Linked to invoices
Stock movements (ledger entries)	33,494	Sales, purchases, returns, adjustments
Payments	11,870	UPI 48%, cash 34%, card 18%
Products at or below reorder level	16	3 of them not yet ordered

CHAPTER 6

IMPLEMENTATION

6.1 INTRODUCTION

The system is implemented as a TypeScript monorepo with three packages: the web application (apps/web), the API (apps/api) and the shared library (packages/shared). The implementation proceeds through the following stages, each of which is a working, tested increment of the system:

1. **Scaffolding:** repository structure, TypeScript configuration, linting and formatting, Docker Compose with a MongoDB replica set, continuous integration and a health-check endpoint.
2. **Data models and seed data:** Mongoose schemas for all collections with validation, the ER design, and the deterministic one-year data generator.
3. **Authentication and roles:** sign-in, short-lived access tokens, rotating refresh tokens, role permissions, user management and an audit log.
4. **Catalogue:** products, categories and suppliers with GST-aware validation, live price breakdown and barcode lookup.
5. **Stock ledger and adjustments:** the single service through which stock changes, stock counts and write-offs, stock history and inventory valuation.
6. **Purchase orders:** drafts, placing orders, partial deliveries, weighted-average cost, cancellation and closing short.
7. **POS billing and GST invoice PDF** (in development).
8. **Returns and customers** (in development).
9. **Dashboard, reports and exports** (in development).
10. **AI modules:** forecasting engine, restock plans with owner confirmation and alerts; then the Gemini assistant (in development), followed by end-to-end testing and deployment.

6.2 IMPLEMENTATION DETAILS

1. Authentication and role-based access

Users sign in with email and password. Passwords are stored as bcrypt hashes with cost factor 12. On success the API returns a JSON Web Token (HS256) valid for 15 minutes, which the browser keeps only in memory, and sets a refresh token in an httpOnly, SameSite=Strict cookie valid for seven days. Only a SHA-256 hash of the refresh token is stored. Each time the token is used it is replaced by a new one; if an old token is ever presented again, the whole chain of sessions is revoked, which protects against stolen

cookies. Login attempts are rate-limited (10 failures per account per IP address in 15 minutes). Every request re-loads the user, so a deactivated account or a changed role takes effect at once. Admins cannot remove their own access, and the last active admin cannot be removed.

2. Catalogue management

Products carry a SKU, optional barcode (with EAN/UPC check digit validation), category, brand, unit (pieces, packs, kilograms, litres and others; fractions are allowed only for loose goods), HSN code, GST rate from the current slabs (0, 0.25, 3, 5, 18 and 40 per cent), cost price, selling price (tax-inclusive or exclusive), optional MRP, reorder level and quantity, and a preferred supplier. While the user types, the form shows a live breakdown of the price before GST, the GST amount and the margin. Suppliers are stored with a GSTIN whose state code and check character are verified, and with a delivery lead time. Cashiers can view products but never see cost prices.

3. Stock ledger and adjustments

All stock changes go through one function, recordMovement (Fig 4.4), which runs inside the caller's transaction. It performs one atomic update that only succeeds if enough stock remains, rounds quantities of loose goods to three decimals in the database, and writes a stock movement with the type, quantity, resulting balance, unit cost, reason, user and source document. A concurrency test sends eight simultaneous sales of three units against ten units in stock and confirms that exactly three succeed. On this basis, managers can record stock counts (entering the counted quantity; only the difference is recorded) and write-offs with a reason, for several products at once and all-or-nothing. The stock history page lists every movement with filters, and the inventory overview values stock at cost and selling price by category and lists items that need attention.

4. Purchase orders

A purchase order can be saved as a draft and edited, or placed straight away. Orders are numbered per Indian financial year in the format PO/26-27/00001, within the 16-character limit for GST documents. GST is calculated per line from the product's rate, as CGST + SGST for a supplier in the shop's state or IGST otherwise, and the expected delivery date defaults to today plus the supplier's lead time. The "Add low-stock items" button fills the order with the supplier's products that are low or out of stock and not already on order. Goods can be received in several deliveries with the supplier's bill number; receiving more than remains is refused, and two people recording the same delivery at once cannot double-count it. Each receipt adds stock through the ledger and updates the product's cost to the weighted average (Fig 4.5). Untouched orders can be cancelled, and partly received orders are closed short.

5. Point of sale and GST invoice (in development)

The billing screen is designed around the reusable product picker already built, which supports keyboard search and barcode scanners. It will add items to a cart, apply discounts before tax, calculate CGST + SGST or IGST from the customer's state, round the total to the nearest rupee, accept cash, UPI, card or a split payment, and save the sale and its stock movements in one transaction. The invoice will be numbered per financial year (INV/26-27/00042) and printed as a PDF showing the shop's GSTIN, HSN codes and the tax breakup. The shared invoice calculation functions are already implemented and tested.

6. AI forecasting, restock plans and assistant (in development)

The forecasting engine and restock planner will implement the methodology of Section 3.3 inside the API, with no external service or cost. Restock plans will be stored as drafts with their items grouped by supplier, and a separate confirmation endpoint, requiring the purchasing permission and called only from the owner's button, will create the purchase orders. The Gemini assistant will be given tools such as sales summary, top products, low stock, forecasts, create_restock_plan and update_restock_plan, but no tool that confirms a plan, so the AI can never place an order by itself.

7. Quality assurance

The code uses TypeScript in strict mode, ESLint and Prettier. Pure logic (GST, money, dates, validation) is covered by unit tests; API endpoints are tested with Supertest against an in-memory MongoDB replica set, including transactions and concurrency; React pages are tested with Testing Library. GitHub Actions runs all checks and builds the Docker images on every push.

Table 6.1: Implementation Status of the Modules

Module	Status	Automated tests
Scaffolding, CI, Docker	Completed	4
Data models and seed data	Completed	35
Authentication, roles, users	Completed	62
Products, categories, suppliers	Completed	47
Stock ledger and adjustments	Completed	32
Purchase orders	Completed	27
GST, money, dates and invoice calculations (shared)	Completed	28
POS billing and GST invoice	In development	–
Returns and customers	In development	–
Dashboard, reports, exports	In development	–

Module	Status	Automated tests
AI forecasting and restock plans	Designed	–
AI assistant (Gemini)	Designed	–

6.3 BLOCK DIAGRAM

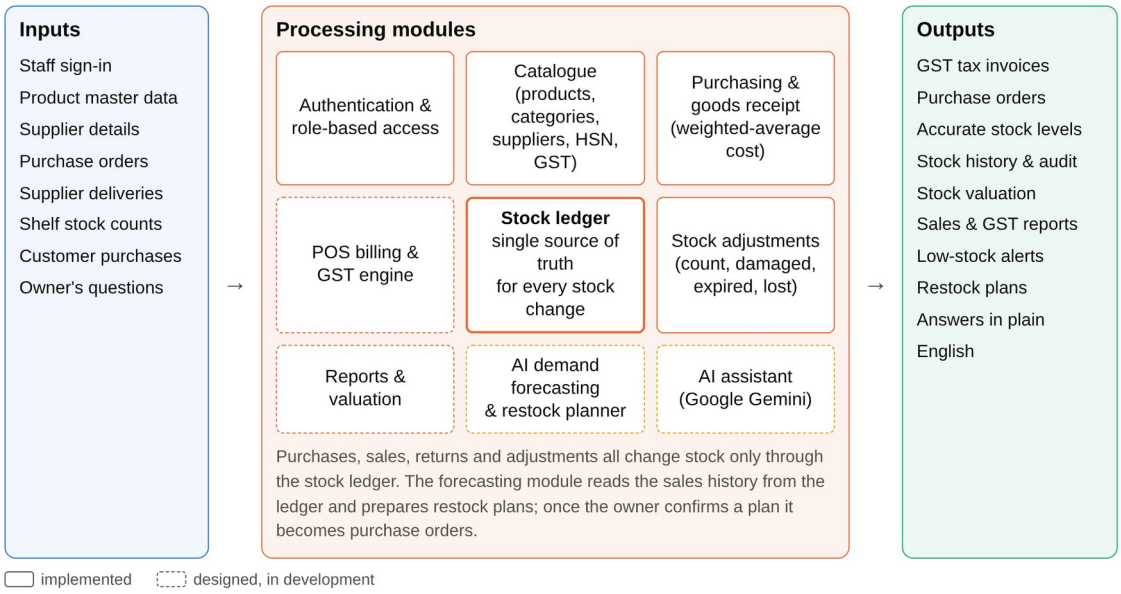


Fig 6.1: Block Diagram of the System

Fig 6.1 shows the main blocks of the system. Inputs from staff, suppliers and customers are processed by the modules in the middle. The stock ledger is the central block: purchasing, billing and adjustments all change stock only through it, and the forecasting module reads the sales history from it. The outputs are GST invoices, purchase orders, accurate stock figures and reports, alerts and restock plans.

6.4 SYSTEM WORK FLOW

Fig 6.2 shows the life cycle of a purchase order as implemented. An order starts as a draft or is placed directly; once placed it can no longer be edited, only received, cancelled, or (when partly received) closed short.

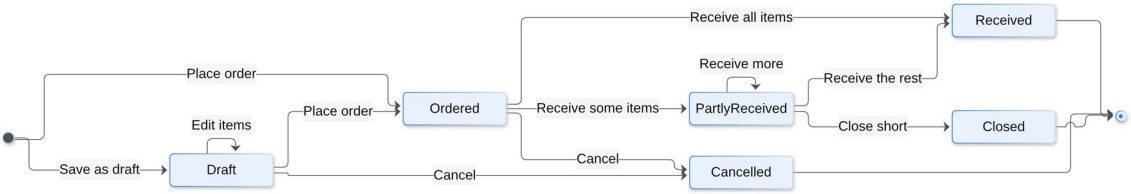


Fig 6.2: Purchase Order Life Cycle

Fig 6.3 shows the sequence of operations when a manager records a delivery. All database steps between the start and the commit of the transaction either complete together or are rolled back together.

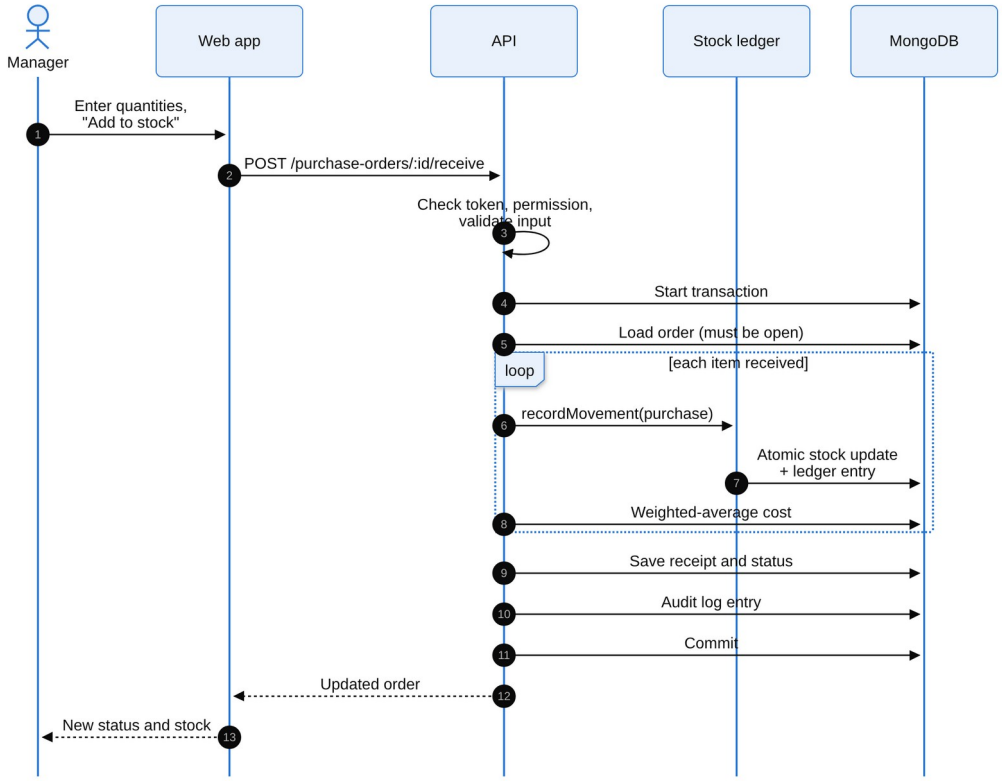


Fig 6.3: Sequence Diagram – Receiving Goods

Fig 6.4 and Fig 6.5 show the designed workflow of the AI restocking agent. A restock plan is a draft until the owner confirms it; the assistant's tools can create and edit drafts, but confirmation is a separate endpoint that only a person can call.

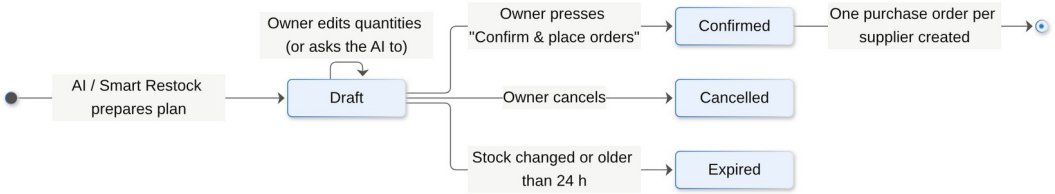


Fig 6.4: Restock Plan States (Design)

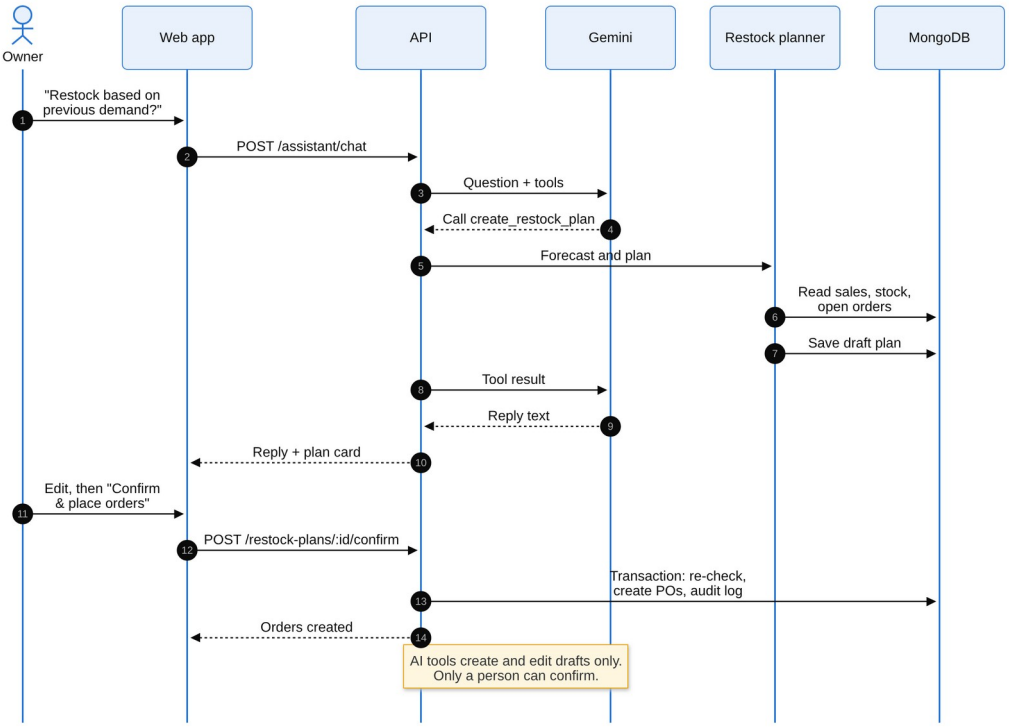


Fig 6.5: Sequence Diagram – AI Restock with Owner Confirmation (Design)

CHAPTER 7

TESTING AND RESULTS

7.1 TEST CASES AND EXECUTION

Testing was carried out at three levels: unit tests for the shared calculation and validation logic, API tests that send real HTTP requests to the server backed by an in-memory MongoDB replica set, and user-interface tests that render the React pages. Table 7.1 lists representative test cases from the automated suite, with the actual result of the latest run.

Table 7.1: Test Cases and Execution

Description	Input	Expected Output	Actual Output	Status
GST within the state	Taxable ₹100.00 at 18%, same state	CGST ₹9.00 + SGST ₹9.00, total ₹118.00	As expected	Pass
GST across states	Supplier in Maharashtra, shop in Karnataka	IGST at the full rate, no CGST/SGST	As expected	Pass
GSTIN validation	GSTIN with a wrong check character	Rejected with a field error	Rejected	Pass
Login with wrong password	Valid email, wrong password	Same "invalid credentials" message as an unknown email	As expected	Pass
Login lock-out	10 failed attempts	Further attempts rate-limited	Blocked (429)	Pass
Refresh token reuse	Replay an old refresh token	Whole session family revoked	Revoked	Pass
Role restriction	Cashier opens stock adjustment	Access denied (403)	Denied	Pass
Negative stock prevented	Remove 15 units when 10 are in stock	Refused, nothing saved	Refused (409)	Pass
Concurrent sales	8 parallel sales of 3 units, stock 10	Exactly 3 succeed, stock 1	Exactly 3 succeeded	Pass
Stock count	Counted 8 when the system shows 10	Movement of −2 recorded, stock 8	As expected	Pass
Partial delivery	Receive 100 of 144 ordered	Status "partly received", 44 on order	As expected	Pass
Over-receipt	Receive more than outstanding	Refused with field error	Refused	Pass
Weighted cost	10 units on shelf at ₹500, receive 6 at ₹530	Stock 16, cost becomes ₹511.25	As expected	Pass

Description	Input	Expected Output	Actual Output	Status
Duplicate SKU	Create product with existing SKU	Error shown on the SKU field	As expected	Pass

Each test case is executed automatically. For example, the weighted-cost test creates a product, places and receives two orders at different prices, and checks the resulting cost price, stock and ledger entries in the database. The concurrency test starts eight sales at the same moment against the same product and checks that no more units are sold than were in stock. The interface tests check, among other things, that cashiers never see cost figures and that a filter cleared by the user does not reappear.

7.2 RESULTS

The complete automated suite of 235 tests passes: 64 unit tests in the shared package, 122 API tests and 49 user-interface tests (Fig 7.1, Table 7.2). The same checks run on GitHub Actions for every change, together with linting, type checking and the production and Docker builds. In addition, every module was exercised manually in a real browser on the one-year demonstration dataset; for example, a purchase order of ₹21,168 was placed, received in two deliveries and checked against the stock, cost and ledger in the database.

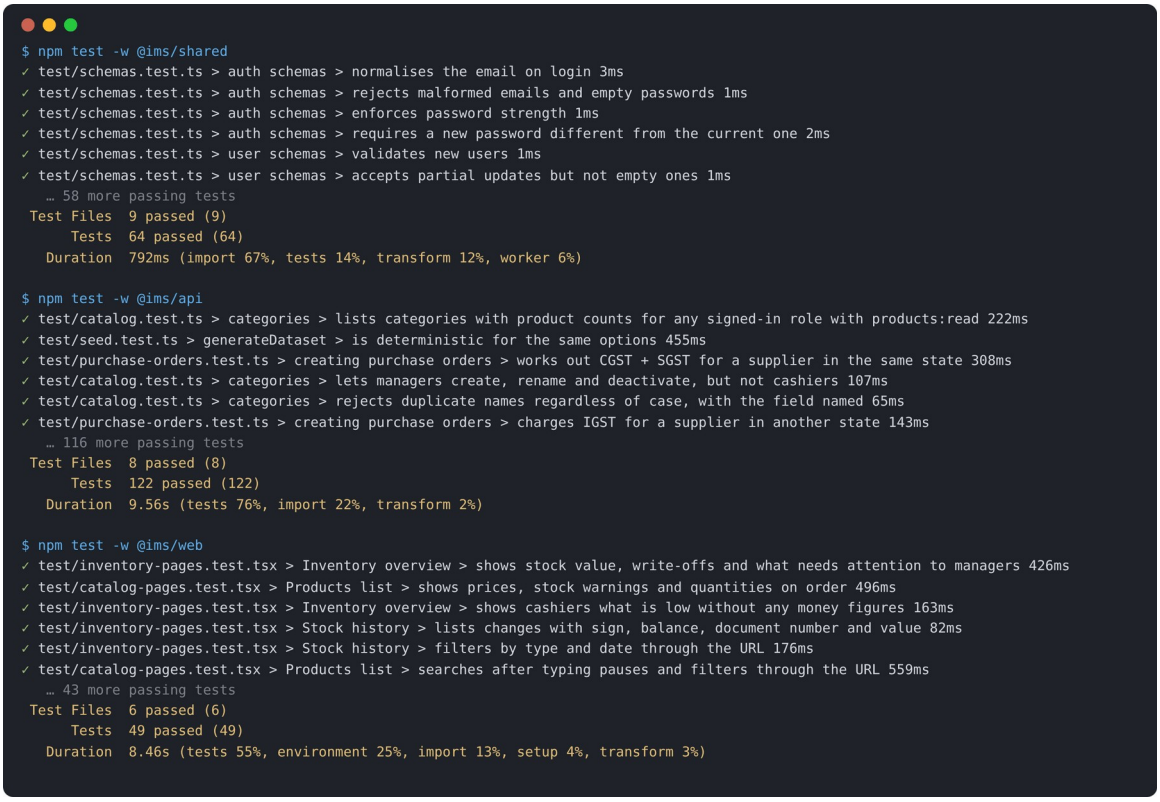


Fig 7.1: Automated Test Run

Table 7.2: Summary of Automated Tests

Test suite	Files	Tests	Result
Shared logic (GST, money, dates, schemas)	9	64	All passed
API (Supertest + MongoDB replica set)	8	122	All passed
Web interface (Testing Library)	6	49	All passed
Total	**23**	**235**	**All passed**

The following sections present the screens of the implemented modules, captured from the running application with the demonstration data.

7.3 LOGIN PAGE

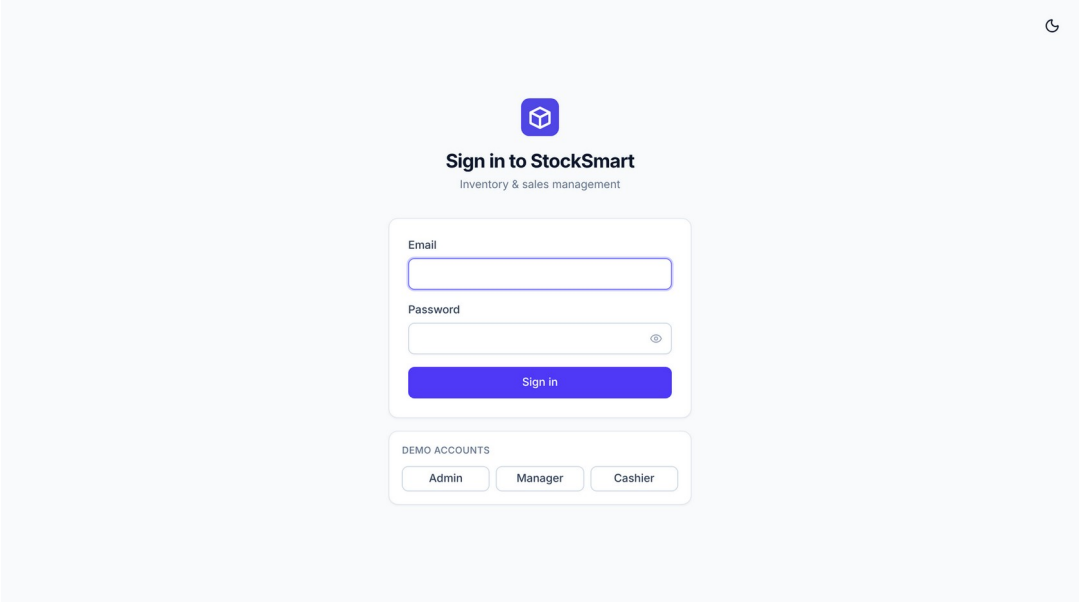


Fig 7.2: Login Page

From Fig 7.2, the login page asks for the email and password and shows clear messages for invalid input. For the demonstration, buttons fill in the demo accounts of each role. After signing in, the user returns to the page they originally asked for, and the menu shows only the modules their role allows.

7.4 PRODUCT CATALOGUE

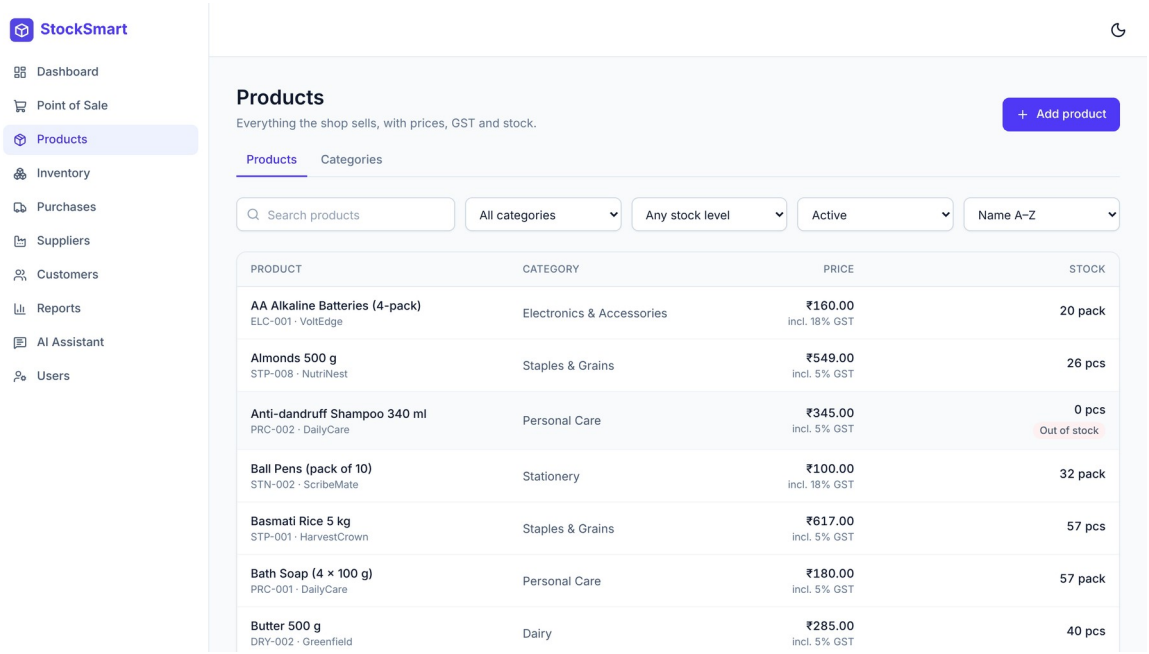


Fig 7.3: Products List

Fig 7.3 shows the product list with search, category, stock-level and status filters. Each row shows the price with its GST rate and the stock with a status badge (low stock, out of stock) that combines colour, an icon and a label, plus the quantity already on order.

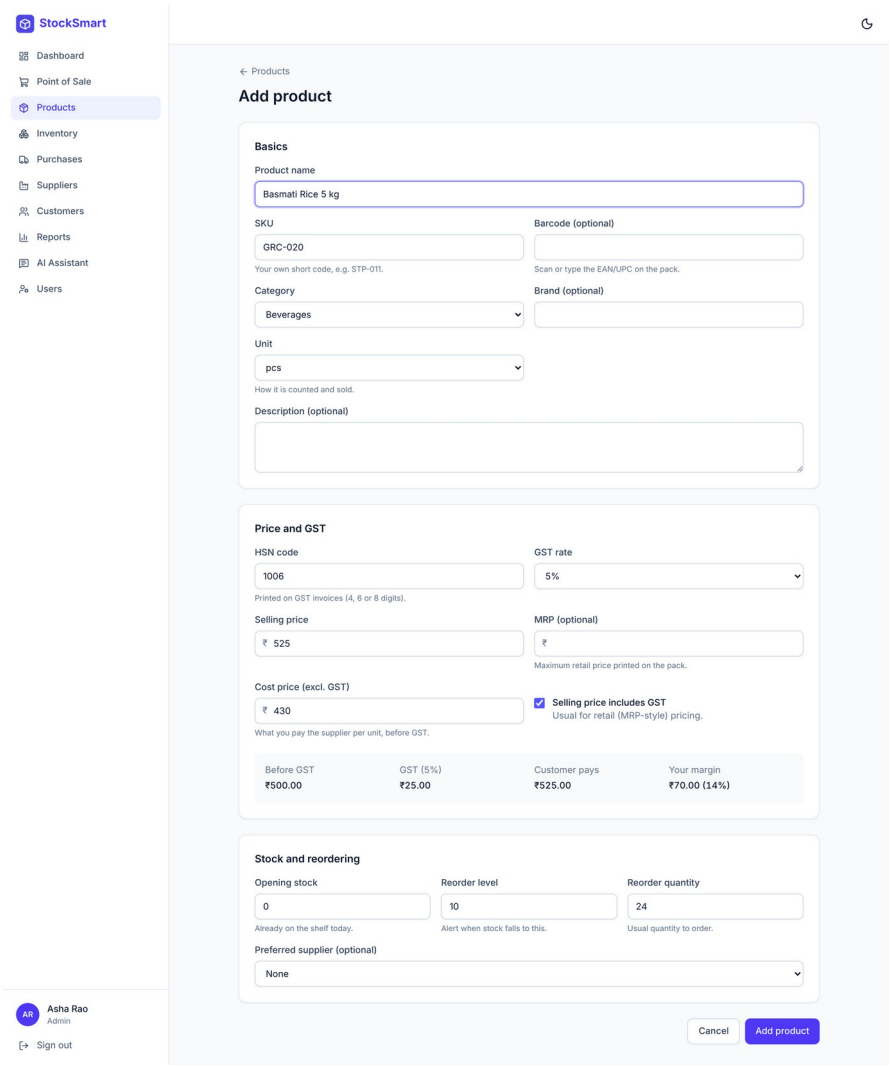


Fig 7.4: Add Product Form with Live Price Breakdown

Fig 7.4 shows the product form. As the selling price, GST rate and cost price are entered, the price breakdown on the right shows the price before GST, the tax and the margin. Barcodes, HSN codes and the MRP rule are checked before the form is sent.

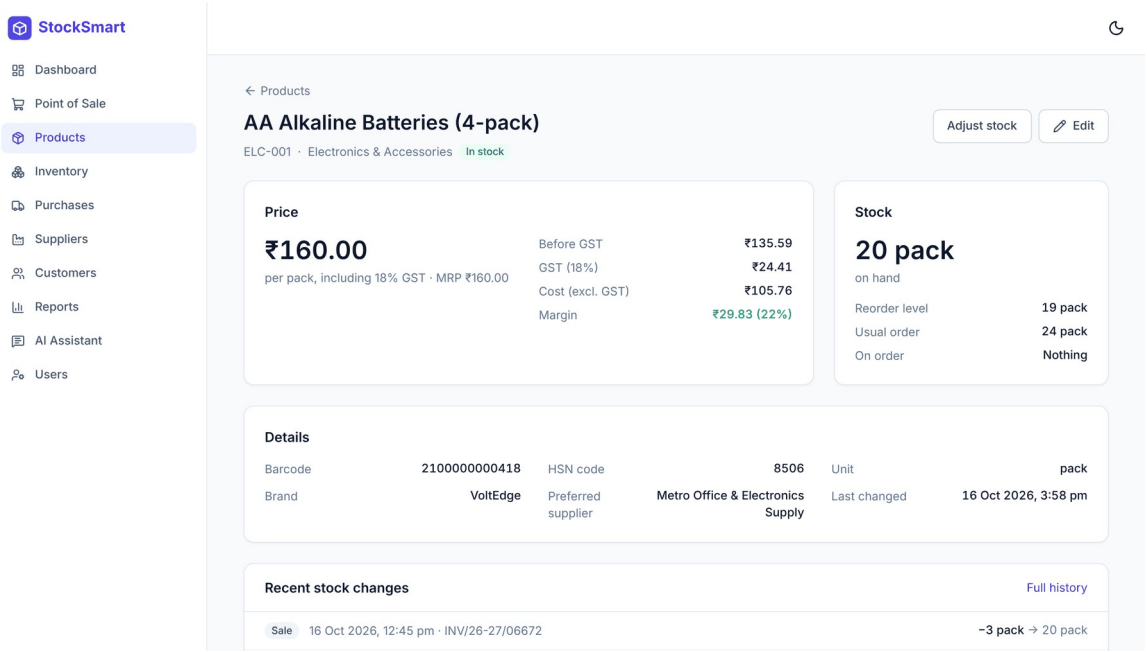


Fig 7.5: Product Details with Recent Stock Changes

Fig 7.5 shows a product's details: price breakdown, stock with reorder level and quantity on order, and the most recent stock movements with the document that caused each one.

7.5 INVENTORY

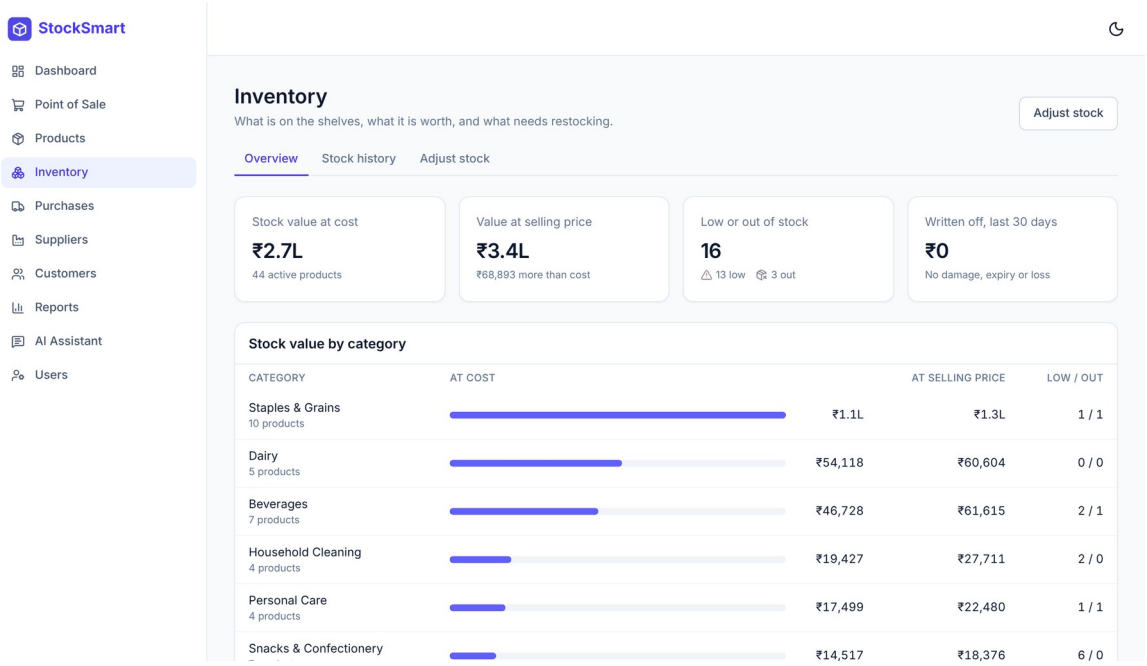


Fig 7.6: Inventory Overview

Fig 7.6 shows the inventory overview: the value of stock at cost and at selling price, the number of low and out-of-stock products, write-offs in the last 30 days, the stock value per category, and a "needs attention" list that shows whether each item is already on order.

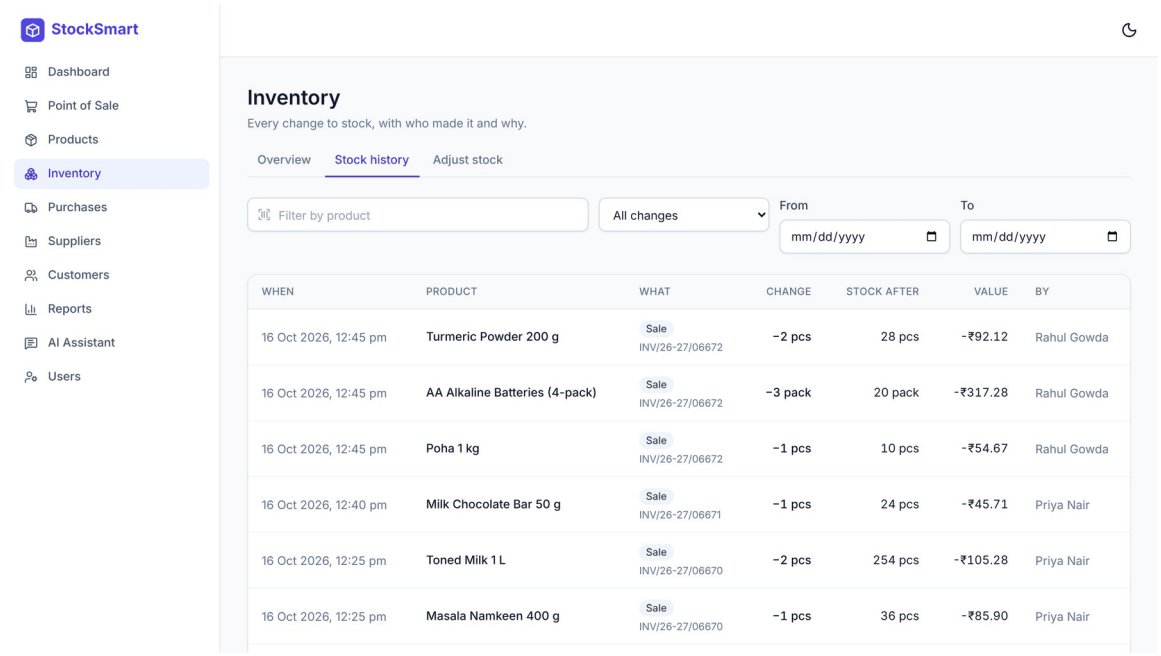


Fig 7.7: Stock History (Stock Ledger)

Fig 7.7 shows the stock history: every change with its date and time, product, type, invoice or order number, change in quantity, resulting stock, value and the user who made it, with filters by product, type and date.

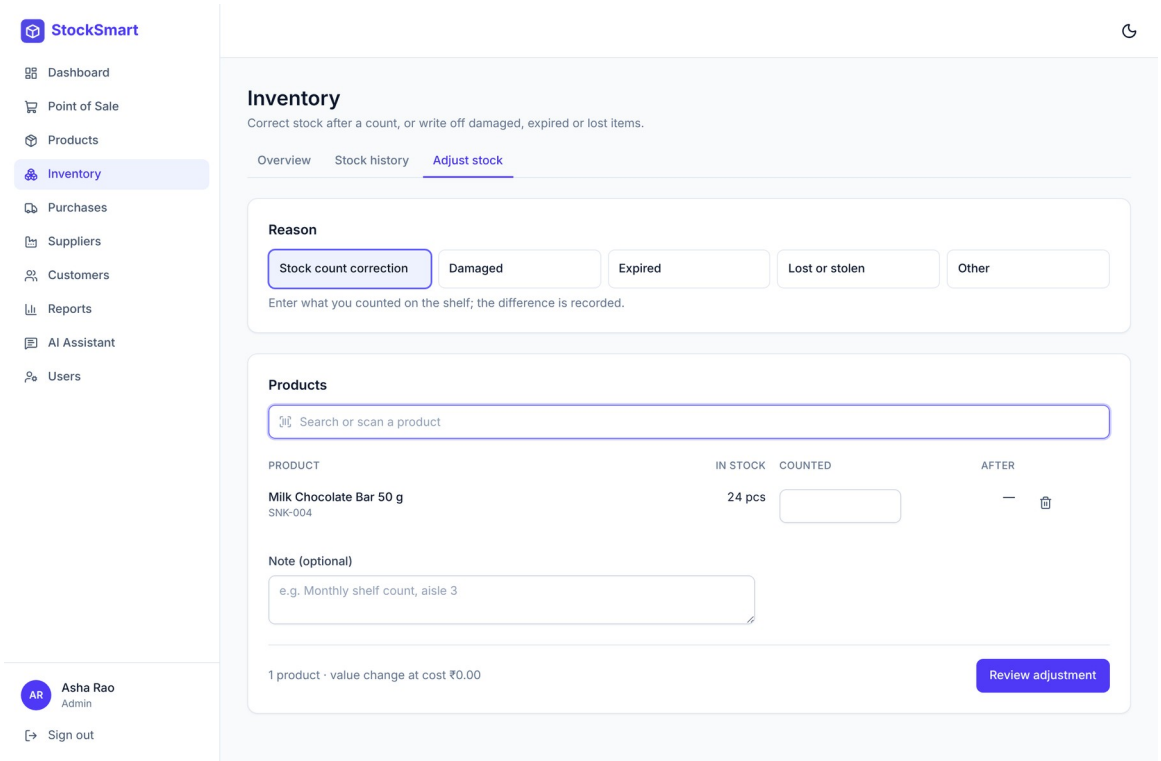


Fig 7.8: Stock Adjustment

Fig 7.8 shows the stock adjustment page. The user chooses a reason (stock count, damaged, expired, lost or other), adds products by search or barcode, and enters the counted

or removed quantity; the page shows the stock before and after and the change in value before the adjustment is saved.

7.6 PURCHASE ORDERS

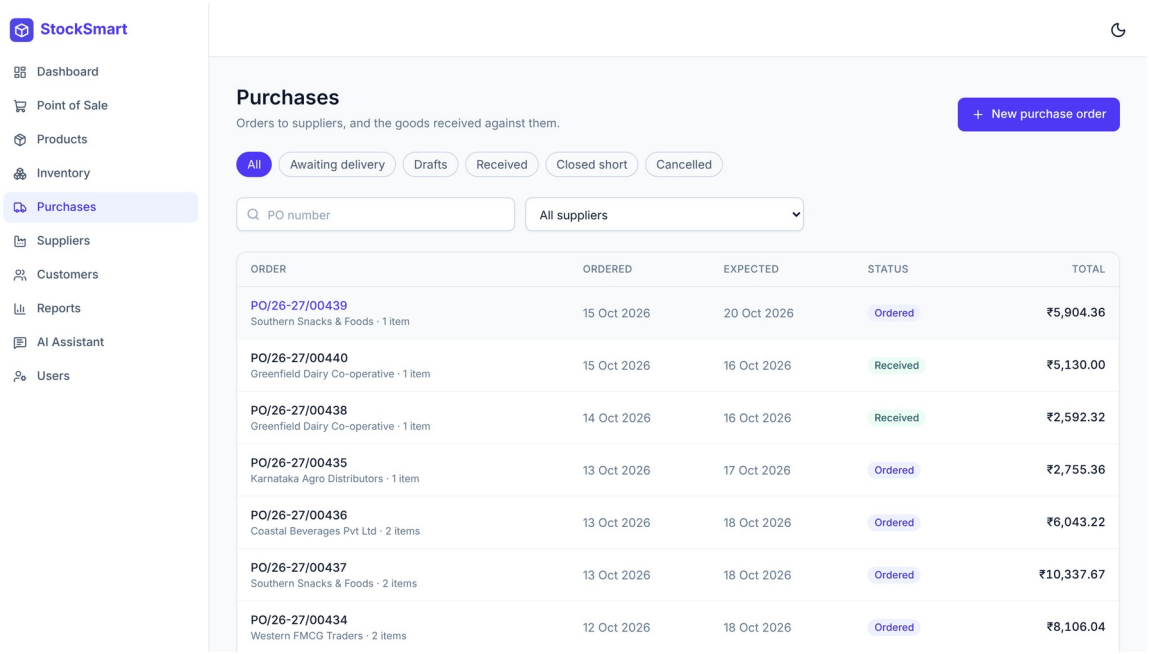


Fig 7.9: Purchase Orders List

Fig 7.9 lists purchase orders with status chips (awaiting delivery, drafts, received, closed short, cancelled), a supplier filter and search by order number. Overdue orders are flagged.

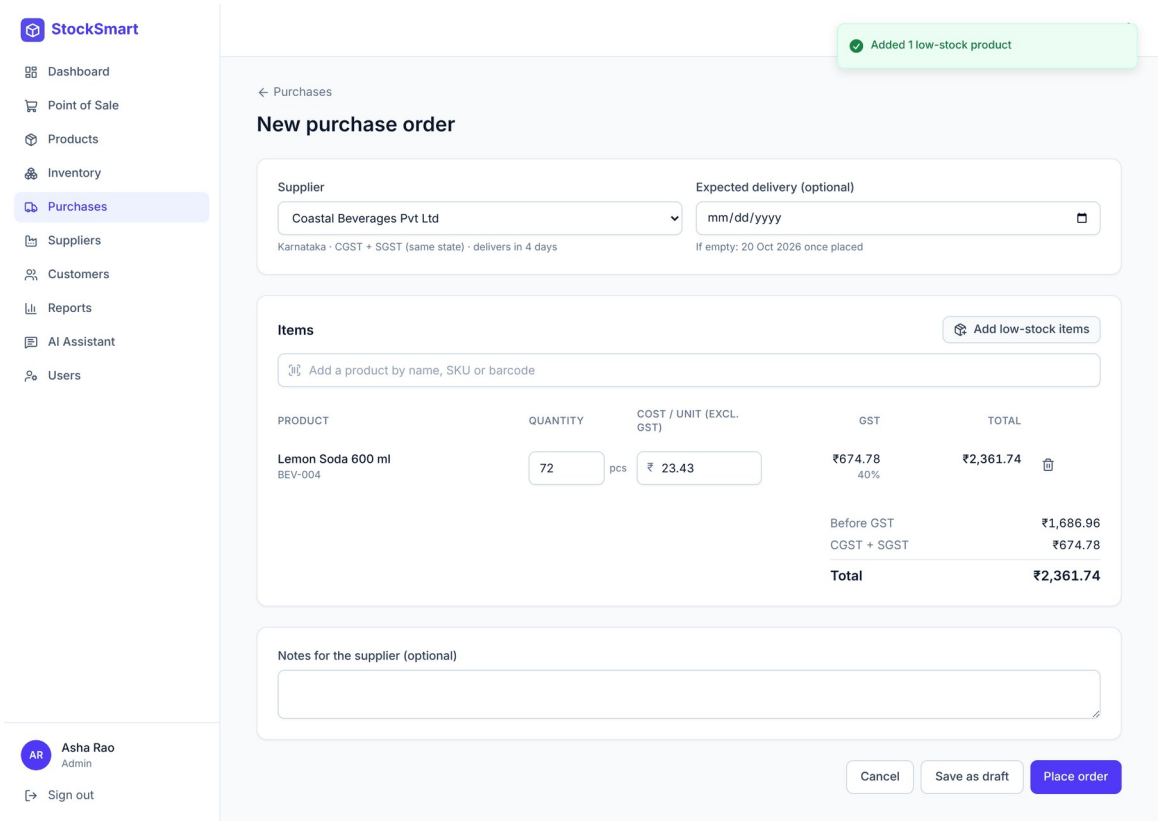


Fig 7.10: New Purchase Order with Live GST

Fig 7.10 shows a new purchase order. After choosing the supplier, "Add low-stock items" adds the supplier's products that need restocking; costs are filled in from the products, and the totals show CGST + SGST or IGST depending on the supplier's state.

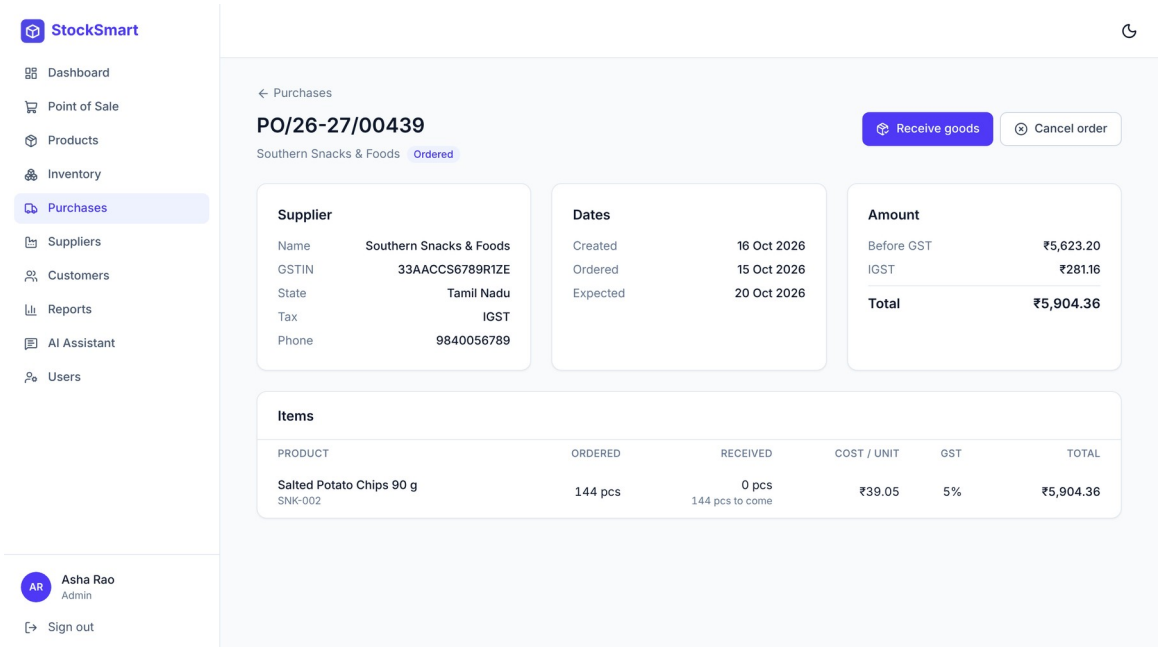


Fig 7.11: Purchase Order Details

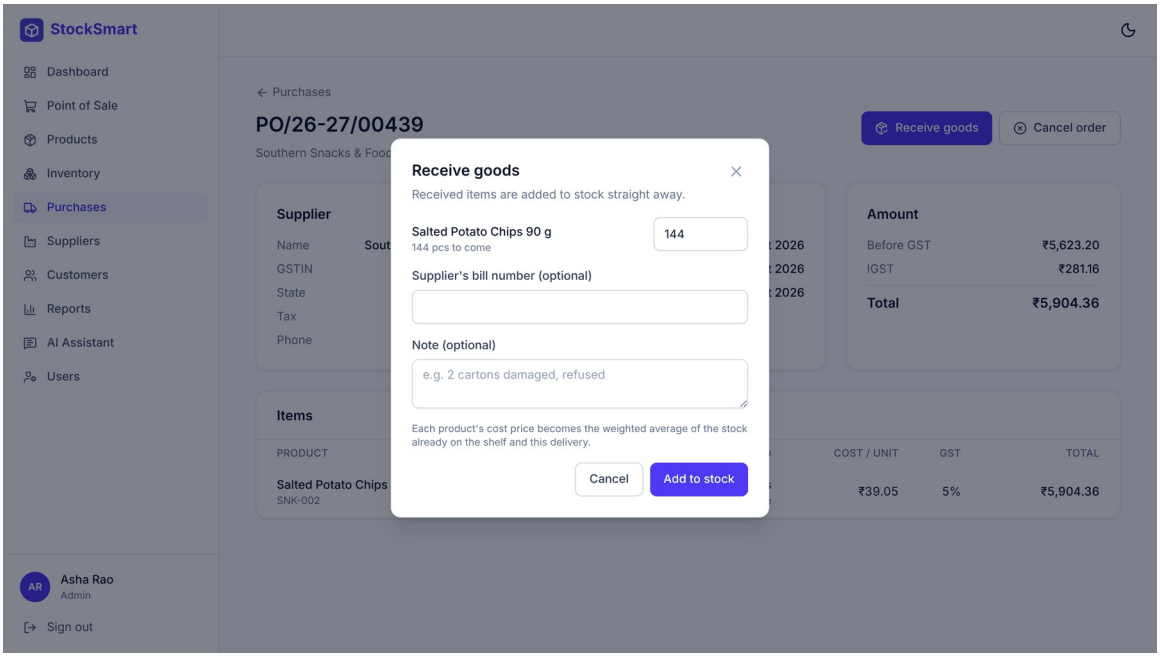


Fig 7.12: Receiving Goods

Fig 7.11 and Fig 7.12 show an order's details and the dialog for recording a delivery. The quantities default to what is still to arrive, and the supplier's bill number can be recorded. Received goods are added to stock immediately.

7.7 SUPPLIERS, USERS AND ROLE-BASED ACCESS

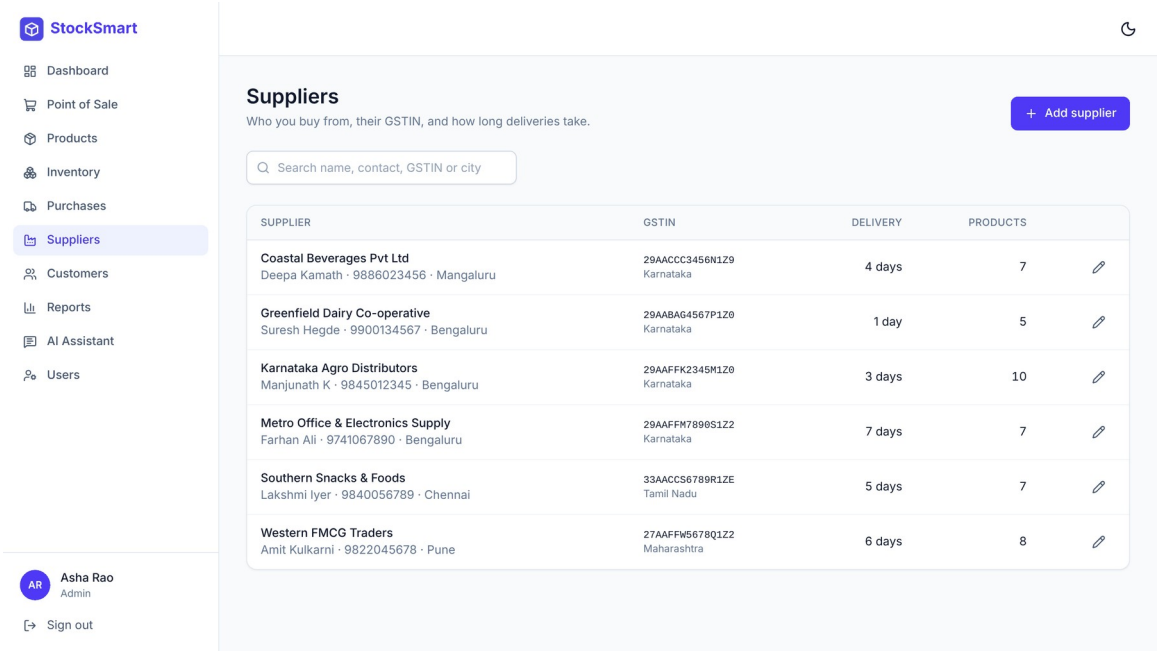


Fig 7.13: Suppliers

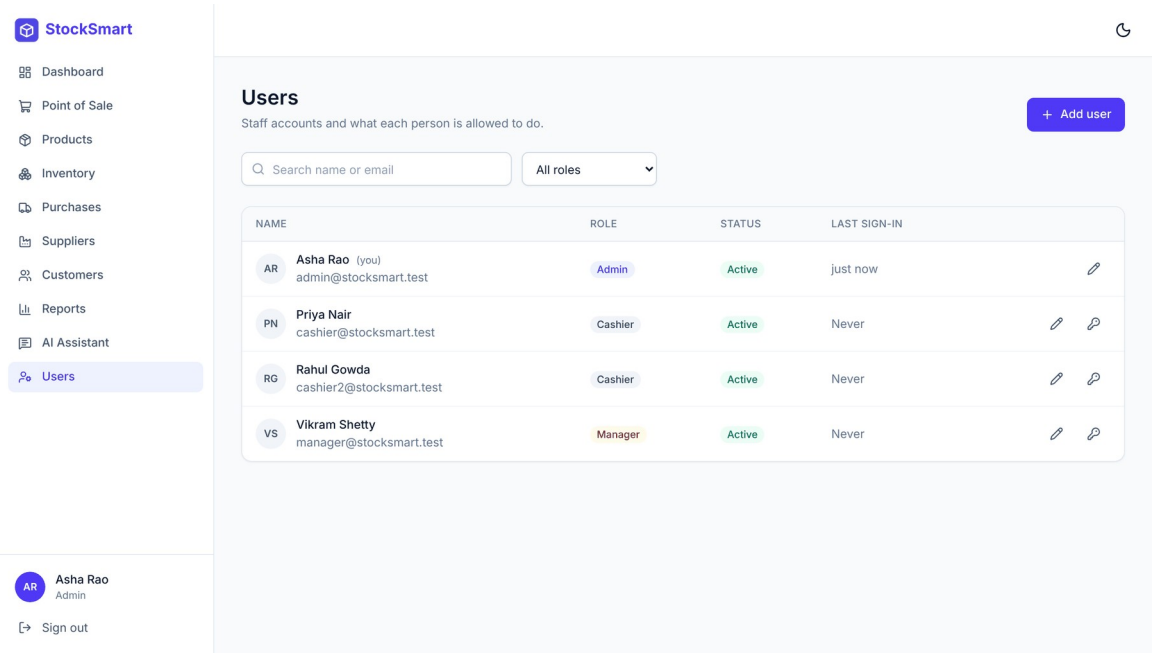


Fig 7.14: User Management (Admin)

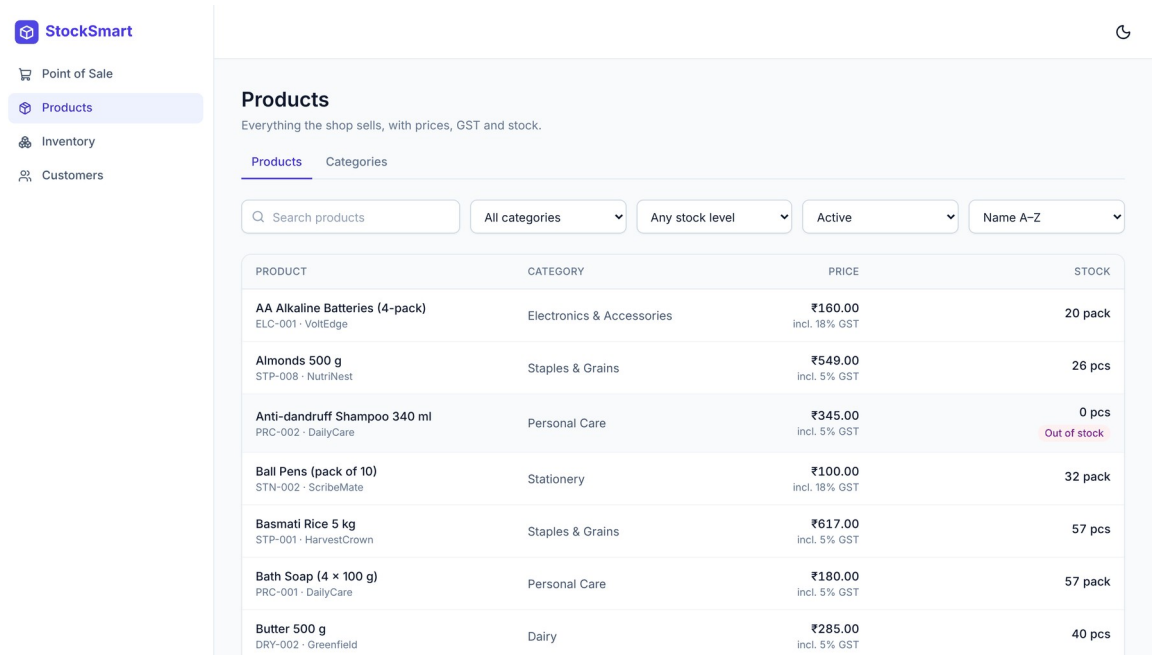


Fig 7.15: Cashier's View of Products

Fig 7.13 shows the suppliers with their GSTIN, state, delivery lead time and number of products. Fig 7.14 shows the user management page, available only to the admin, with roles, status and last sign-in. Fig 7.15 shows the same product list as seen by a cashier: the menu contains only the modules a cashier may use, and editing controls and cost information are hidden.

7.8 RESULT SUMMARY

The implemented modules work as intended and are verified by 235 automated tests, all passing, as well as by manual testing in the browser on a year of realistic data. Stock remains exactly consistent with its ledger under concurrent use; GST is calculated correctly for supplies within and across states; purchase orders handle partial deliveries and update costs correctly; and each role sees and does only what it is allowed. The data analysis of Chapter 5 confirms that the demand patterns of a retail store (weekly cycle, seasons and festivals) are strong and product-specific, which supports the forecasting approach designed for the AI restocking agent. The remaining modules (POS billing, returns, reports and the AI features) build on this tested foundation.

CHAPTER 8
CONCLUSION AND FUTURE ENHANCEMENT

This project presents a Digital Inventory and Sales Management System for small and medium retail stores in India. Built on the MERN stack with TypeScript, it brings the product catalogue, suppliers, stock, purchasing and billing into one secure web application. Its central design decision, a transactional stock ledger through which every change of stock must pass, keeps the recorded stock exactly consistent with the history of what was bought, sold and written off, even when several staff work at the same time. The system follows Indian GST rules throughout, from HSN codes and GSTIN validation to the CGST/SGST/IGST split and financial-year document numbering, and it controls access by role with modern token-based security.

The user management, catalogue, stock ledger, stock adjustment and purchase order modules are complete and verified by 235 automated tests. Analysis of a year of simulated trading shows strong weekly, seasonal and festival patterns in demand, which justifies the design of the AI restocking agent: a Holt-Winters forecasting engine with festival uplift and safety stock that prepares restock plans, with the owner reviewing and confirming every order. Together with the planned Gemini assistant, this applies AI as decision support that saves the owner time while keeping control of the shop's money in human hands.

The immediate next steps are to complete point-of-sale billing with printable GST invoices, customer returns, the dashboard and reports, and the two AI modules, followed by end-to-end testing and deployment. Further enhancements that would extend the system include:

1. **Multiple branches:** stock transfers between stores and consolidated reports.
2. **Supplier integration:** sending confirmed purchase orders to suppliers by email or WhatsApp.
3. **E-invoicing and GST returns:** generating IRN-ready e-invoices and GSTR-1/GSTR-3B summaries.
4. **Advanced forecasting:** comparing Holt-Winters with gradient-boosted trees or neural models once real sales data is available, and using price and promotion information.
5. **Offline billing:** a progressive web app that keeps billing during internet outages and synchronises later.
6. **Batch and expiry tracking:** for pharmacies and food items, with alerts before goods expire.
7. **Customer loyalty:** points, purchase history and personalised offers.

8. **Voice queries and regional languages:** asking the assistant in Kannada, Hindi or other Indian languages.

In conclusion, the system offers a practical, affordable and reliable foundation for digitising a retail store, with AI features designed to help the owner make better restocking decisions without taking those decisions away from them.

REFERENCES

[1] F. F. Agboola, Y. M. Malgwi, M. A. Mahmud, and J. P. Oguntoye, "Development of a web-based platform for automating an inventory management of a small and medium enterprise," FUDMA Journal of Sciences, vol. 6, no. 5, pp. 57–65, 2022, doi: 10.33003/fjs-2022-0605-1064.

[2] H. Q. Lim, A. R. Rahmat, and S. Suban@Ani, "An inventory management system for small businesses," Journal of Digital System Development, vol. 3, no. 1, pp. 29–44, 2025, doi: 10.32890/jdsd2025.3.1.3.

[3] P. Phuthaikampanatpinya and C. Dechsupa, "A point-of-sale platform for the retail small and medium enterprises," Academic Journal of Science and Technology, Dhonburi Rajabhat University, vol. 1, no. 1, 2023.

[4] H. R. Patil, S. R. Kotgire, N. S. Madhekar, P. N. Shelke, and I. Raskar, "Account + GST inventory management ERP system (using cloud storage)," International Journal for Research in Applied Science and Engineering Technology (IJRASET), vol. 10, no. 4, 2022, doi: 10.22214/ijraset.2022.41898.

[5] M. Nasseri, T. Falatouri, P. Brandtner, and F. Darbanian, "Applying machine learning in retail demand prediction—A comparison of tree-based ensembles and long short-term memory-based deep learning," Applied Sciences, vol. 13, no. 19, Art. no. 11112, 2023, doi: 10.3390/app131911112.

[6] V. Pasupuleti, B. Thuraka, C. S. Kodete, and S. Malisetty, "Enhancing supply chain agility and sustainability through machine learning: Optimization techniques for logistics and inventory management," Logistics, vol. 8, no. 3, Art. no. 73, 2024, doi: 10.3390/logistics8030073.

[7] R. N. Boute, J. Gijsbrechts, W. van Jaarsveld, and N. Vanvuchelen, "Deep reinforcement learning for inventory control: A roadmap," European Journal of Operational Research, vol. 298, no. 2, pp. 401–412, 2022, doi: 10.1016/j.ejor.2021.07.016.

[8] L. Shi, Z. Tang, N. Zhang, X. Zhang, and Z. Yang, "A survey on employing large language models for text-to-SQL tasks," ACM Computing Surveys, vol. 58, no. 2, Art. no. 54, 2025, doi: 10.1145/3737873.

[9] J. Jiang et al., "SiriusBI: A comprehensive LLM-powered solution for data analytics in business intelligence," Proceedings of the VLDB Endowment, vol. 18, no. 12, pp. 4860–4873, 2025, doi: 10.14778/3750601.3750610.

[10] C. Heins, "Artificial intelligence in retail – a systematic literature review," Foresight, vol. 25, no. 2, pp. 264–286, 2023, doi: 10.1108/FS-10-2021-0210.